Lab Notebook

Alicja Brzezowska

2026-06-01

Lab 1: Preliminary Simulation Visualization

Date: 30/10/2025

Notes

- Visualising outputs of the Flamingo simulations using CSD3. Project data onto 2D (along the z axis);
- Visualising both slices through the simulation, and projections of all data
- Do both dark matter and baryonic gas; try to see the difference of gas distribution vs that of dark matter, cannot zoom in that much: resolution is not good enough

Lab 2: tSZ Theory

Date: 03/11/2025

Notes

The tSZ effect is caused by the inverse Compton scattering of cmb photons on energetic ICM electrons. It induces a variation of the apparent brightness of the CMB towards any line of sight that passes through a reservoir of hot plasma. This variation is given by **Ruppin2019PressureProfile**:

$$\frac{\Delta I_{tSZ}}{I_0} = y f(\nu, T_e) \tag{1}$$

$$y(\hat{n}) = \frac{\sigma_T}{m_e c^2} \int P_e dl \tag{2}$$

$y = \int P_e \, \mathrm{d}\chi$, with $P_e = n_e T_e$ leads to degeneracy: high n_e / low T_e vs. low n_e / high T_e. Need both $n_e(r)$ and $T_e(r)$ to break degeneracy; vary physical parameters in fits.

Hadzhiyska, B., "Evidence for large baryonic feedback at low and intermediate redshifts from kinematic Sunyaev-Zel'dovich observations with ACT and DESI photometric galaxies"

Notes

[methods]

- used ACT CMB maps and galaxy data from DESI's photometric Luminous Red Galaxy catalog
- stacked the signals from the galaxies
- measured how much CMB was distorted by the electron moving with those galaxies: informing about the gas distribution
- : co

Notes

[results]

- gas extends farther than the dark matter halos
- previous Illustris simulation, which had stronger feedback mechanisms, matched the data better; current IllustrisTNG predicts less extended gas

Lab 3: XGPaint Pseudocode - beginning

Date: 04/11/2025

Summary

Original and upgraded pseudo code for XGPaint.

HEALpy tutorial

Notes

- θ (colatitude, measured from north pole)
- ϕ (longitude)
- Pixels organised into horizontal rings (constant colatitude), in 1d array [the RING ordering]. A ring may be shifted by half a pixel in ϕ.
- NSIDE: defines the resolution of the map, in powers of 2

Summary

Healpix maps are 2D on the sphere but stored in a 1D array $[p_0, p_1, p_2, \dots]$. Ring 1 has 2 pixels, etc. Pixel areas are equal; higher colatitudes have fewer pixels to preserve area.

Wills XGPaint pseudocode - Boryana's overview

Notes

1. Create a grid of integrated properties, e.g. integrated $y(\theta, M, z)$. tSZ measures P_e (electron/gas pressure); Compton-y maps from P_e plus frequency response; rSZ not included. kSZ measures ρ_e (electron density).
2. Build an interpolator from the grid.
3. Loop over each halo in the catalogue: a). Identify all pixels within $N\,\theta_{200}$ of each halo centre. b). For each pixel, compute angular distance from the centre and assign the interpolated value.

Notes on the original XGPaint pseudocode:

Notes

- The input data is the WebSky simulations: HDF5 file: x,y,z, M

Upgraded XGPaint pseudocode - Boryana's overview

Notes

1. Precompute the profiles and build an interpolator as in the original code.
2. Build a tree from all the halo+field particles.
3. Query the tree out to N R_200 at each of the halo center positions to determine which particles belong to which halo. This can now be arranged into a large, contiguous array of indices with nstart and nlength, giving you a way to parse through it.
4. Compute the distances between the particles and the halos (I believe that there is a way to do this in the above step, but I am not completely certain)

5. Given the distance of each particle associated with a halo and the target profile for that halo, compute how much the particle needs to contribute such that the radially averaged profile is preserved.
6. Paint on pixels using TSC or nearest-grid point simultaneously for all particles

Model of the code:

Julia vs Python performance

Julia is faster, as:

- JIT compilation: just-in-time compilation, compilation: compilation (bytecode translation into machine code) during the execution of a program, rather than before execution.
- type specialization
- native threading
- array operations

Python by itself is inherently slower:

- interpreted
- dynamic typing overhead
- global interpreter lock limits threading

To speed up Python, use JAX. JAX is a library for accelerator-oriented array computation and program transformation, designed for high-performance numerical computing and large-scale machine learning. JAX has features that directly correspond to Julia's speed up:

- JIT compilation
  `jax.jit` ("just-in-time" compilation) is used to accelerate numerical computations by transforming ordinary Python/JAX functions into optimized compiled code. Instead of executing operations step-by-step through the Python interpreter, JAX first *traces* the function using symbolic placeholders for the inputs, records the mathematical operations as a computation graph, and then passes this graph to the XLA compiler (Accelerated Linear Algebra). XLA generates highly optimized machine code for the available hardware (CPU/GPU/TPU). The first call to a jitted function is typically slower because compilation occurs at that stage, but subsequent calls reuse the compiled version and are therefore much faster.

JAX: does not operate on pythons tuples: instead operates on arrays which have a strictly defined dimension;

input data

- The original version of XGPaint uses WebSky simulation outputs: halo catalogs with (M, θ, z)
- The new XGPaint will use AbacusSummit: the

Creating profiles

- tSZ profiles based on **Battaglia_2016**
- flatLambdaCDM from astropy.cosmology: inputs: Hubble constant; Omega matter density (in units of the critical density at z = 0), temperature of the CMB at z= 0; outputs: various cosmological methods, stores all cosmological data in one place.

- flatLambdaCDM: assumes no spatial curvature, assumes that there is some non-zero cold dark matter density (Ω_c), baryon density(Ω_b) and dark energy density: Ω_Λ = 1 - Ω_m; Background theory Ω_i: present day density parameters: ρ_i / $\rho_{critical}$ (i = matter, baryon, cold dark matter), where $\rho_{critical} = 3\ H_0^2\ /\ 8\ \pi$ G
- astropy.cosmology is not compatible with jax: astropy.cosmology was developed for numpy-style operations, not differentiable/ compilable for jax. People developed libraries for compatibility with JAX: **unxt** for unit handling, **wcosmo** for the FlatLambdaCDM cosmology. Wcosmo: does not have ρ_{crit} and angular distance
- Battaglia et.al **Battaglia__2016**, have simulated the distribution of matter with hydrodynamical simulations using varying parameters. The cosmological parameters that reproduce the observed distribution best were used for the flatLambdaCDM input.
- M_{200}: halo mass enclosed within a R_{200} (there, mean density is 200 times the critical density)
- Battaglia et. al. also used these simulations to infer the pressure, density, and temperature profiles of galaxy clusters.
- GNFW: *generalized-Navarro–Frenk–White profile*: gives the density of dark matter as a function of radius fitted to dark matter halos; equation is of the form:..
- goal: find the Compton Y Parameter: as that is the observable; it is the line of sight integral of electron pressure (dimensionless)

Navarro-Frenk-White Profile

One of the most fundamental predictions of cold dark matter (CDM) models is that the density should follow a Navarro, Frenk White (NFW) profile throughout most of the halo **Battaglia2005HaloDispersion**. It is characterized by a density profile that scales with radius as $\rho(r) = \rho_0(r/r_s)^{-1}(1 + r/r_s)^{-2}$, where ρ_0 is a normalization factor and r_s is the scale radius (they vary from halo to halo).

The generalized Frenk-Navarro-White Profile for pressure distribution **Nagai2007ICM**:

$$\frac{P_r}{P_{200}} = \frac{P_0}{x^\gamma(1 + x^\alpha)^{(\beta-\gamma)/\alpha}} \tag{3}$$

where $x = r/r_s$, $r_s = r_{200}/c_{200}$ (is the scale radius), the powers: α, β, γ are the slopes at $r \approx r_s, r >>> r_s, r << r_s$, respectively. The impact of these parameters: $\alpha, \beta, \gamma, c_{500}, P_0$ on the estimates of Ω_m, σ_8 obtained by the analysis of the power spectrum of the tSZ effect has been a subject of previous studies **Ruppin2019PressureProfile**. In this work, we will also fit these parameters, trying to see which values reproduce our data set best. The parameter c_{200}: the concentration parameter: $c = r_{200}/r_s$, is weakly correlated with the mass; also the formation epoch is also strongly correlated with the concentration parameter.

In the original XGPaint code:

Interpolator

Notes

- structure: profile grid function: (create the 3d grid (M, theta, z))
- interpolator is cubic (between interpolated values fits a third power polynomial: matches first and second derivatives at grid points).
- initially use scipy.regulargridinterpolator, method = "cubic" or method = "linear"; however the cubic interpolator didnt work; need to use the linear version: wondering if the linear is enough: we do inteprolate over the logs which makes the thing actually quite smooth

-
- other types of interpolators
- for profile grid: in Python, use meshgrid instead of the nested for loops, which are optimized to work in Julia. also vectorize all operations –> see a 20 fold speed-up in python compared with the nested for loop.

Coding notes

3.9.1 terminal commands - python

- changing python env in conda: conda activate name
- getting path in Finder: right click + option
- running code in terminal: python file-path

3.9.2 terminal commands - julia

- REPL initial set-up: 1.] activate: "file-path", 2. backspace using XGPaint
- running code: include(file path)

Lab 4: XGPaint Tree

Date: 14/11/2025

Summary

Pseudocode and notes for the Tree querying code for tSZPaint

notes

1. input data: abacussummit simulation RA, DEC, Chi, healpix: in a lightcone: for different z values
2. input data for tests: empty healpy map, with random integers per pixel
3. use KDTree: for normal Python
4. problems to implement the tree with jax:
   (a) as jax jit works when the shape of arrays in the function is static: independent of the content of inputs: which by nature is not possible with a tree-based algorithm
   (b) people have used PyTorch K-NN built-in function for performance-critical tasks: use a library to call PyTorch's knn, and be able to JIT the code: https://github.com/rdyro/torch2jax
   (c) also recently JAX k-D: which is quicker on NVDIA GPUs: but slower than Scipy on CPU;
   (d) vector-data base vs tree:
5. pseudocode for python implementation:
   (a) 1. build fake data: random map of nofpixels per pixel

   github repository

   (a) $_{i}nit_{.py:better for imports; creates a python module}$
   (a)

Lab 5: tSZPaint benchmarks

Date: 17/11/2025

integration schemes

5.1.1 quad

5.1.2 Abel Transform

Abel Transform $F(y)$ of a function $f(r)$ is the projection of a circularly symmetric function along the line of sight.

$$F(y) = 2\int_0^\infty f(\sqrt{x^2+y^2})dx = 2\int_{|y|}^\infty f(r)\frac{rdr}{\sqrt{r^2-y^2}} \tag{4}$$

Lab 6: ideas for the future of this masters

Date:

1. what if we were to use a proabibilistic approach of fitting the pressure profile: instead of using the NFW profile and varying parameters: let the whole functionality of the function vary (see Nikhils paper on the neutron start EoS from mergers)

Lab 7: Globus: downloading data

Date: 26/01/2026

Note to self for Globus:

setting up Globus: Need to have the Globus app installed on my local computer (download from their site): do this for linux!!! not for macbook. I.e go on this website:

"https://docs.globus.org/globus-connect-personal/install/linux/" and follow the instructions.

Once installed: u need to go into globus everytime!: "$ cd globusconnectpersonal-x.y.z"

Need to connect to globus CLI: done using ./globusconnectpersonal -setup –no-gui

login in: created a new collection: Input a value for the Endpoint Name: rds registered new endpoint, id: 67e0a8b0-fac8-11f0-8072-0e56852e4f6b setup completed successfully

also u need to have the globus app running on your macbook so that u get the rds file folder to run; just double click on the app icon; and it will appear in the top corner of your screen (no app window pops up);

as u have this downloaded already: now simply need to do cd globus.. and make sure to switch the app on: and then done; can transfer

cd /home/ab2927

cd globusconnectpersonal-3.2.8

./globusconnectpersonal -start

./globusconnectpersonal -status

collection format:

have two collections: STARONE for healpix maps and the other one for the halo maps

Lab 8: Coordinate transformations

Date: 25/01/2025

HEALPix: θ and ϕ on a unit sphere:

1. HEALPix to cartesian:

$x = sin(\theta)cos(\phi)$ $y = sin(\theta)sin(\phi)$ $z = cos\theta$

2. R_{200} to θ_{200}:

R is the actual distance in Mpc; whereas θ is the angular size of R as seen from far away (?);

convert using diameter distance!: $\theta_{200} = arctan(R_{200}/D(z))$

D is obtained via the astropy module for a flat lambda cdm: $\propto \frac{R(z)S(\chi)}{(1+z)}$

3. Chord length from θ_{200}:

When calculating the distances between points on the query tree; then we want to have the chord length: for two point separated by alpha on the unit sphere we have a chord length of:

$L = 2sin(\alpha/2)$; here α is $N\theta_{200}$

Lab 9: Painting function - on mock data

Date: 26/01/2025

Painting code

code structure

The file for generating the Y-Compton maps is composed of the following functions; in order:

1. Generate mock data catalogs;

HEALPix maps are given with θ and ϕ: on the unit sphere

2. Load Interpolator;

3. Build tree;

-convert the coordinates to cartesian (tree only works in cartesian) - build a tree from ll the HEALPix pixels

4. Query tree;

The function queries the tree to up to N times the θ_{200} value using the query-ball-point function (takes the sampling point as the center of a certain radius, and selects the points within the radius ball as the neighboring points)

Walk through the function:

- for each halo convert the angular distance to 3D Euclidean distance (chord length) on the unit sphere using the equation: $L = 2sin(\theta/2)$ Question to myself: how come cord length is what we want? -use the build tree function with the particles: the pix-xyz is a list which contains all particles in their natural order alongside their xyz coordinates - query the tree to find all the pixels within the search radius of each halo - create a nested list with pixel indices for each halo: then flatten the list (subset of the pix-xyz list with different ordering) - the list format is particle index, x, y ,z:

5. Weigh particle contr

inputs: particle$_c ounts and corresponding pixel indices$

-wrapper to perform the initial weighing - get particle counts for each pixel - calculate the raw asymmetry weights: $w_p \propto N^{5/3}$

-then put compute weights numba to perfom the normalization

6. Compute weights numba:

[inside the weigh particle contr]

inputs: the raw weights from the weigh particle contr function

outputs: weighted weights (normalized)

- create radial bins from 0 to N times θ_{200} (rn its 20 radial bins): this is for the centers of pixels **QUESTION**: should i somehow include the pixel edges or is that negligible? - bin pixels by distance - normalize weights in each bin -returns the weights for - prange: and numba decorator; parallel large runs, runs on multiple CPU cores -raw weights: non-normalized weights: the weights which just have proportionality of power 5/3 to the particle count from the dark matter only simulation

7. Paint y-vectorized:

QUESTION TO MYSELF: What is the given information about the halos from our data: do we get the masses, redshifts, and radial coordinates or not? **NOTE TO MYSELF**: given the mass and redshift of a halo,: its size and angular size are determined via the provided cosmology.

-firstly build the tree from pixels -convert halo coordinates to cartesian from spherical -query the tree for all halos - compute the asymmetry weights -convert all: masses; thetas etc to log scales for the interpolator - interpolate the y values for the given masses of halos

Painting asymmetrically

For increased accuracy, we want to preserve the asymmetry of the signal. In order to correctly paint asymmetrically, we note that the preserved physical quantity in tSZ is the energy. For our signal, we can get the energy by multiplying the Y Compton signal times the area it occupies. Fortunately for healpix, by definition, all pixels have the same area, so the total energy for N pixels is simply proportional to $Y_i \cdot N_{\text{pixels}}$. We are interested in how the Y_i Compton parameter signal changes around halo centers, so we need to consider a thin annulus between θ_i and θ_i+1. The assigned weights for each pixel, which preserve the overall magnitude of the signal, are given by:

$$Y_p = \frac{Y_i \cdot N_{\text{pixels}} \cdot w_p}{\sum_{\text{pixels}} w_p} \tag{5}$$

where Y is $\propto M^{5/3} \propto N^{5/3}$, $w_p = N_p^{5/3}$. We can check that this recovers the correct value of Y in the sense that $Y_i = \frac{\sum_{\text{pixels}} Y_p}{N_{\text{pixels}}}$

Comparison with XGPaint

Using the same mock map I have painted the signals with XGPaint and TSZPaint comparing their outputs.

Computation time: The Julia Code for now proves to be faster by 1.15x times compared to Pure Python.

Results: I have compared the values obtained by XGPaint and TSZPaint through measuring the Pearson correlation coefficient between Python and Julia Pixel values; firstly do that for the unweighted version: as then can directly compare the outputs (symmetric painting); if see that the wieghted versions prove to be different: then we know that it is the weighting that we expect to cause the qualitative differences

cannot compare weighting for the mock data: as it is not properly normalized: the masses of halos which are randomly distributed

QUESTION: systematic error for weighting: given our data format: in the halo files we have the halo masses indirectly: from number of particles and the mass per particle; in the healpix files: we like assign

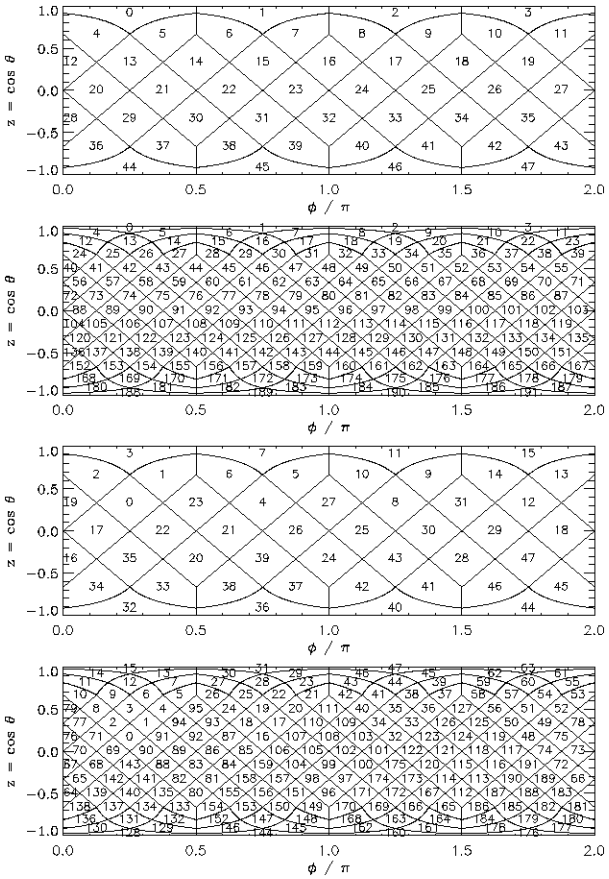


Figure 1: Cylindrical projection of the HEALPix division of a sphere and two natural pixel numbering schemes (RING and NESTED) allowed by HEALPix

Lab 10: Abacussummit data format

Date: 27/01/2026

asdf - advanced scientific data format

Lightcone0 healcount maps

healpix format: have 8192^2 times 12 pixels

healpix background theory: we have 12 base pixels: in three rings around the poles and equator;

The healpix ordering in the abacus simulation suite is nested:

NESTED VS RING ORDERING:

Ring Scheme: simply count the pixels moving down from the north pole to the south pole along each iso-latitude ring; it is the ring scheme which makes the spherical harmonic fourier transforms easy to implement

Nested Scheme: One can arrange the pixel indices in twelve tree structures: which correspond to the base resolution pixels; This can be easily implemented since, due to the simple description of pixel boundaries: the analytical mapping of the HEALPIx base resolution elements: curvelinear quadrilaterals into a [0,1]x[0,1] square exists. This tree structure allows one to implement efficiently all applications involving newarest-neighbour searches

the coordinates are ra, dec

This is because at the coarsest resolution, nside = 1: have 4 pixels north pole, 4 pixels south pole and 4 pixels on the equator; then as the nside is increased, they get divided further

for each file: range of redshifts: i.e if have steps 22 - 24: will have a redshift for each step **QUESTION**: what do i take as the mean redshift?

for steps 0306-308 blah: the redshift is like 2.16 for steps 0605 blah the redshift is like 0.7

Halo maps

in halo$_{i}nfoasdf$: $listofhalosandtheirpositionsinacomovingvolume$:

the coordinates are in a box: x,y,z

we are also given the amount of particles in each halo N;

What is the threshold through which we determine the edge of the halo to count the particles

in the header file: 1. cosmology model (with omega M, omega B etc) 2. the mass of particle in units of Sun Masses and units of Sun masses per small h; 3. the redshift 4. the corresponding comoving radial distance to the snapshot redshift measured in Mpc/ h : $\chi(z)$ 5. The box size in Mpc and Mpc/h 6. The position of the observer of the simulation: LCO at the origin

h is the dimensionless hubble parameter: $h = \frac{H_0}{100kms^{-1}Mpc^{-1}}$

masses of the halos: get given all the particles in each halo: from which we can calculate the masses of the halos; as know the mass of the particles;

spherical overdensities: give sizes of halos:

QUESTION: how do i map the box format data which contains halo information onto the healpix data which contains particles? **QUESTION**: how do i ensure that the mass in my halo is the correct mass in the pixels in the halo

NOTE TO SELF: the mollview image displayed when plotting 8192 times 8192 is not the full resolution: the resolution is limited by the amount of pixels on the laptop screen

creating tszpaint to work on real data

currently: the mock data has:

1. one file with particle counts in healpix: this is consistent with real data; but also need to extract the mass per like value of the pixel and the overall redshift of out lighcone

2. a file with halo data: which includes the healpix positions of the halo centers; and then also include the masses of the halos;

need to match the healpix lightcones to the halo catalogs to identify the correct pixels to correct halos:

PROBLEM:

cannot think of a way to map exactly the correct redshifts: as given redhshift ranges for the healpix lightcones and the halo catalogs comoving volumes; these dont match;

PROBLEM:

how do i deal with edge cases?

PROBLEM:

how do i ensure that the overall mass in my healpix catalog matches that of the comoving volume

Lab 11: Using the HPC

Date: 01/02/2026

theory

11.1.1 RAM Memory

Our local computers have limited RAM (8 - 16 GB) and our task needs 20-30 GB; whereas the HPC node has around 300 GB total RAM.

Nodes are like "computers" in the CPU cluster; Cores are single processing units inside the CPUs: one CPU has many cores: each core can execute its own instruction stream (i.e. one thread at a time, ignoring hyperthreading)

11.1.2 Parallelization

Additionally, using the HPC allows us to split the computing into multiple cores

submitting a job into the hpc

sbatch /home/ab2927/my$_s lurm_s ubmit.sh$

cclake has 304 GB of RAM;

to check the job status in the queue: squeue -j number of query

to check the stats of the job; how long it took etc: sacct -j number of query –format=JobID,Submit,Start,End,Elaps

for the slurm job: this codes main concern is with the amount of memory needs to load at once when loading and querying the tree; therefore allow large memory: 200 GB; but use only a couple of cores in the CPU node: like 8:

firstly need to change this line: SBATCH –cpus-per-task=8

but also add these lines: export OMP$_N UM_T HREADS = 8 exportNUMBA_N UM_T HREADS =$ 8

for resetting a wrong git commit comment: git reset –soft HEAD 1 goes one commit before: but keeps changes git reset unstages everything

to only stage files which have been modified or deleted: no untracked files (useful for our situation when there is a lot of newly created files such as outputs of simulations/ hpc files):

git add -u

and then commit normally etc

When doing commits from different places to the same repository, like the hpc and the local computer:

git pull –rebase=true

if doesnt work because of conflicts: then git merge

token for the hpc for github:

github$_p at_1 1BO4NHLQ0CvlIwQzHrfwP_N Kn3Oz4ICaAlUGKNxnfXpAgvzY5O2kLPMcc9CtYcSXY63A$

Lab 12: 02/02/2026

Date: tSZPaintpseudocode

y-profile-jax.py

paint.py

12.2.1 create-mock-particle-data

12.2.2 create-mock-halo-catalogs

12.2.3 convert-rad-to-cart

12.2.4 compute-theta-200

12.2.5 load-interpolator

12.2.6 build-tree

inputs: nside of the HEALPix map (which determines the number of pixels on the healpix map)

functionality:

1. convert nside to the number of pixels (using healpy); 2. obtain the pixel indices; 3. from pixel indices obtain their angular positions on the healpix map (using healpy) 4. convert angular positions to cartesian using `convert-rad-to-cart` 5. build a cKDTree from the cartesian coordinates

outputs: tree of pixels, pixel cartesian coordinates, pixel indices

12.2.7 query-tree

inputs: cartesian coordinates of halos; theta-200 of halos; particle-tree; pixel cartesian coordinates, N (value determining the times theta-200 which we want to query particles till);

1. Transform the search angles (N times theta-200) into distances on the tree: chord distance between particles (search-radii) 2. query the tree using ball point with these distances to obtain which pixels belong to which halos: create a list of total numbers of particles in each halo : pix-in-halos; which is a nested list (list of the pixel indices in each halo, and a list of all halos) 3. flatten the array: i.e have just pixel indices in order for each halo 4. create a neighbouring array: which maps the index of the halo to the indexes of the pixels: i.e pix-in-halos: [7,14,5,6,23]; and halo-indices": [0,1,1,1,1] etc... 5. also define the lists: halo-start (the number of pixels at which the new halo starts) and halo-count (the anount of pixels in each halo) for weighting

outputs: flattened list of pixel indices belonging to halos; corresponding neihbouring array which maps particles onto halos; distances of pixels from their halo centers;

halo indices (on the healpix); halo-counts (number of halos);

12.2.8 compute-weights

inputs: distances between the halo centers and pixels; halo counts; halo starts; theta 200 for halos; N ; nbins (number of radial bins between theta = 0 to theta = N theta 200);

1. firstly start with all pixels having the same weights; 2. create bin edges using N and nbins; 3. for each halo separately (using prange): obtain the start index of the halo and the counts for each halo; then get the slice of the distances and weights list which corresponds to this halo;

for each particle in this halo: find the bin to which it belongs to; add the particle contribution along with the weight contribution of the particles;

then normalize within bins: divide by the total sum of the values in the bin;

outputs: weights for particles

12.2.9 `compute-initial-weights`

:

inputs:

obtain the proportionality of the particle counts with the

outputs:

12.2.10 `paint-y`

inputs:halo-theta; halo-phi, mass of halos, particle-counts, interpolator

functionality: 1. build and query tree for all particles and halos at once. 2. calculate the weights for each halo using the weighting function; 3. compute all the logs of masses and the computed distances from the center for each halo; interpolate the y values from the grid; then add the interpolated y values to a y-compton map

outputs: y-map

12.2.11 `paint`$_{yc}hunked$

inputs:

outputs:

12.2.12 `compute-particle-contributions`

redshift-finder.py

12.3.1 obtain-healcount-redshift:

inputs: the file directory; the file name; opens the healcount asdf file; extracts the redshift key outputs: the redshift of the file

12.3.2 main

: loops over all the healcount files in the rds folder returns a text file with file name and corresponding redshift

Lab 13: Optimizing the painting

Date: 03/02/2026 :

The current time and memory usage of the code using the chunked painting method:

theta, phi = pixlib.-pix2ang-ring(nside, ipix) Starting: 20000 halos, nside=2048 | mem: 1228 MB particle$_c ounts$: $402.7MB, dtype = int64 build_t ree(npix = 50331648, pix - xyz = 1208.0MB)$: $14.954s|mem : 2346MB y_m ap : 402.7MB[chunk0 breakdown]prep = 0.001s, interp = 0.560s, accum = 0.009s---Timing(200 chunks, 27, 362, 649 pixel-halopairs)---query_t ree : 5.622s compute_w eights : 2.591s interp + accumulate : 142.853s Done : 166.100s|mem : 2981MB$

the largest cost comes from interpolating the values of the y parameter for each pixel;

solved if dont use the chunked painting method, and instead ask for more memory on the node to do vectorized painting all at once

notes from meeting

1. Need to download different files for halo catalogs: in the lightcone format (where the further u go out the "younger" things are) as opposed to snapshots of simulations; 2. Run the painting file on the simulation outputs 3. for the outputs; also create a file which plots the Y parameter with the mass for physical meaning 4. Change the N theta 200 to be the r-90 given in the simulation output; 5. Visualize the painting on the zoomed-in version of the simulation next to the simulation output to see the accuracy of the painting 6. Create a threshold on the halos: only halos more massive than $10^{13} M_{sun}$ should be extracted from the catalog of halos: signal dominated by these more massive halos; run threshold tests to see how much signal the smaller halos get;

Lab 14: Dependencies in the github package

Date: 05/02/2026 :

pyproject.toml:

-> high-level dependencies that the project needs; -> version constraints;

uv.lock:

-> specific, exact versions of the dependencies used in the project;

if the user also has uv: this download of dependencies is easily synced: type uv sync to get all the dependencies;

Lab 15: New lightcone format for halos

Date: 07/02/2026

variables in the halo catalog files:

1. DeltaRedshift: the difference in redshift values across one slice of the simulation; 2. the overdensity parameters: 'SODensity', 'SODensityL1', 'SODensityL2'

python usage reminder lol:

running the tszpaint package using different approaches:

python -m tszpaint.paint to run paint.py file (this way it presents tszpaint as a package to python and not as a file;

cannot do: python /tszpaint/paint.py: but this messes up the imports inside our tszpaint repository: as now should not have from tszpaint.data import blah but instead should have from data import blah

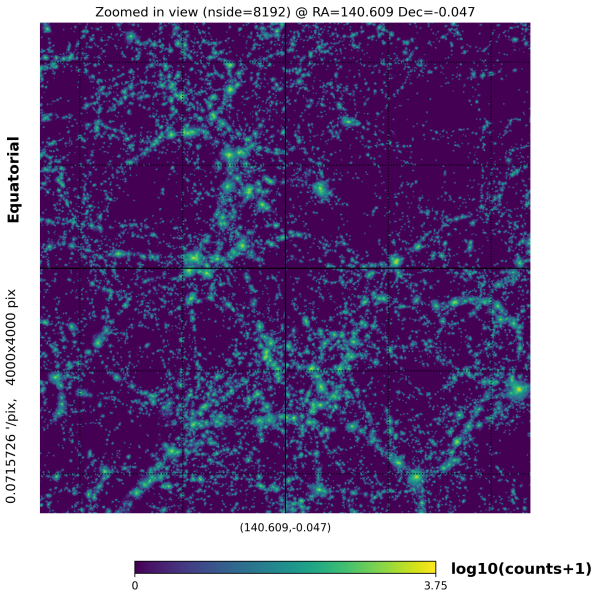


Figure 2: Zoom into the simulation only output from the healpix-count simulation file; centered on the brightest pixel

Lab 16: visualizing the output: zooming in

Date: 08/02/2026

Create zoomed in plots on the same halo (choose brightest signal) for the raw simulation healpix file and the painting file for validation of the painting method;

in order to zoom: in use smoothing effects: oversample and interpolate to get smoother edges:

1. the intrinsic resolution of our data is given by nside: pix-arcmin : hp.nside2resol(nside, arcmin=true)

then do; reso = pix-arcmin / 6.0; xsize = 4000: maqke the output image grid 6 times finer: oversample the projection so that edges are not sharp;

similar to bicubic interpolation in photoshop;

important: check which ordering of healpix is used for visualization: our asdf simulation outputs use the nested ordering

NOTE: check this zooming in tool:

mollzoom([map, fig, rot, coord, unit, ...]); Interactive mollweide plot with zoomed gnomview

Lab 17: Validation of the painting mechanism

Date: 08/02/2026

Performance tests

Physics accuracy tests

GNFW test

Create a plot which tests whether the data follow the predicted GNFW structure for the halos:

a) plot the y-parameter with the distance from the halo center;

b) plot the y parameter with mass of the halo:

pseudocode:

- create a variable which saves the total magnitude of the y signal per halo: sum the number of pixels in each halo times the y signal parameter for that halo

- plot against the mass of each halo: calculated from N-particles times mass particle

function I: get total y:

already have the weighted y values for each pixel; also have the list which allocated which pixels belong to which halo; need to sum these y values for each halo; get the coresponding mass of the halo from the halo catalog; plot

function II; create plot:

–> bin the halos by their masses into N bins; the error in x is the width of the bin; –> for each bin calculate the mean value of the y compton parameter (should theoretically be the integrated compton parameter; but here all healpix pixels have the same area: only shift up or down); alongside with a variance (goes as 1 / sqrt(N)); –> plot on a log-log plot –> linear regression to get the power dependence

1. Check how this changes with the number of bins for each halo evaluation

2. Check how this changes with N $R_{search} = N \times \theta_{200}$

NOTE: Tell Boryana that in the halo files there is no information about the r_{90} for individual halos

NOTE: Figure out how would we use the SO parameter for halo radius queries: right now i dont see how this is relevant: how can one parameter apply to all halos simultaneously?

Halo allocation test

1. Compare visually using the zoomed in image if particles around halos get correctly identified

2.

redshift difference test

:

1. Check deltaz in the data 2. Apply this new extremal z: z + deltaz; 3. recalculate the map

Comparison against xgpaint

1. push these files into xgpaint 2. create a zoom in plotting fucntion 3. create a comparison with xgpaint

Miscellanous

Lab 18: cosmology model issues

Date: 08/02/2026

parameters

code has been written inspired by the julia version xgpaint: using the battaglia profile cosmology; however in the halo files headers: have the parameters used for abacus simulations: should directly use them

use of astropy

look into how effective is the astropy module at calculating different parameters and cosmological values: maybe should switch to doing it yourself

Lab 19: debugging the painting mechanism

Date: 08/02/2026

Issues with the mapping

1. In the list of halos, there seem to be ones that do not correspond to true halos on the healpix map;

This could be due to redshift mismatch between the halo list and the healpix list: some halos which are present at this redshift

The Halo Catalog:

Redshift: 0.541613797989815 DeltaRedshift: 0.002176522865272

In the healpix fil, it contains various redshifts for each step

for the 0671-676 file:

Redshift of healcounts file: (0.543790320855087, 0.541613797989815, 0.539341448739708, 0.537038377222165, 0.534755334274484, 0.532478911187038)

for the 0665-0670 file:

Redshift of healcounts file: (0.557258776368988, 0.554970964882863, 0.552715388051422, 0.550482890384634, 0.548199299096671, 0.545936328767368)

also have this column: 'L1OutputRedshifts': which contains the main stopping simulation redshifts: the same as for the halo catalog files

as the range of the halo file is completely within the 0671-0676 file: then we shouldnt get these Healpix the halo catalog: there should be less of them than we see in Julia particles

to check if this is an effect: load the three neighbouring healpix maps; add them together: and superimpose the single halo catalog

2. The found maximum in the y-compton signal corresponds to a different halo than the found maximum for Healpix pure simulation output

3. Thought about querying the tree for overlapping halos:

A lot of halos come in close pairs or groups: their signal partially overlaps. This shouldnt pose a problem for a radially symmetric painting mechanism, like that used in XGPaint. All information is derived from the halo catalog files, the painted signal gets superimposed in the overlap region, which is correct.

However for the case of asymmetric painting, which uses a weighting scheme, this introduces a systematic error. Now, two datasets are present, the halo catalog as before, with halo mass and position information, and the underlying healpix map. The underlying healpix distribution has a "baked in" overlapping signal already in the pixels. The painting scheme has a weighting scheme built in: we preserve the total theoretically predicted y-compton signal at a distance x from the halo centre using the mass information from the mass catalogs; however we redistribute the weights within the halo to follow the asymmetric pattern in the healpix file. The overlap from two halos creates such an antisymmetry as well! This will artificially boost the asymmetry of the signal for each halo: as the information about the asymmetry is included from the beginning: it takes away weight from the halo and puts more into the bit with the overlap. The redistribution boosts the halo overlap asymmetry.

potential solutions:

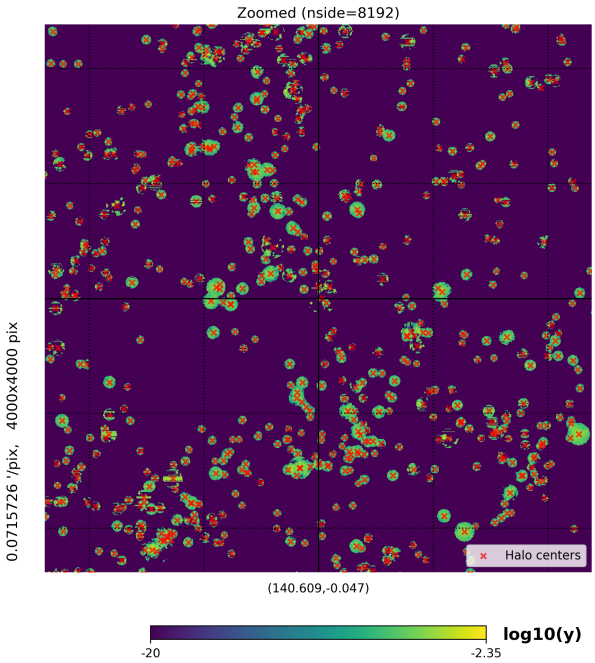


Figure 3: Zoom into the simulation using the painting function with superimposed centers of the halos on them

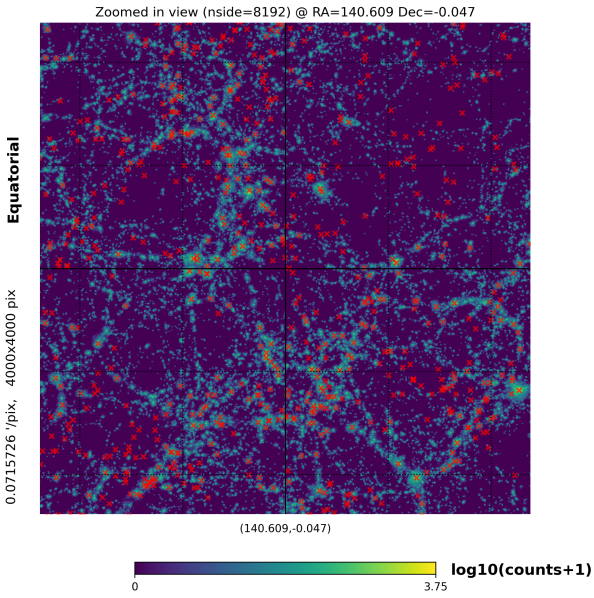


Figure 4: Zoom into the raw simulation healpix output with the halo centers superimposed on it

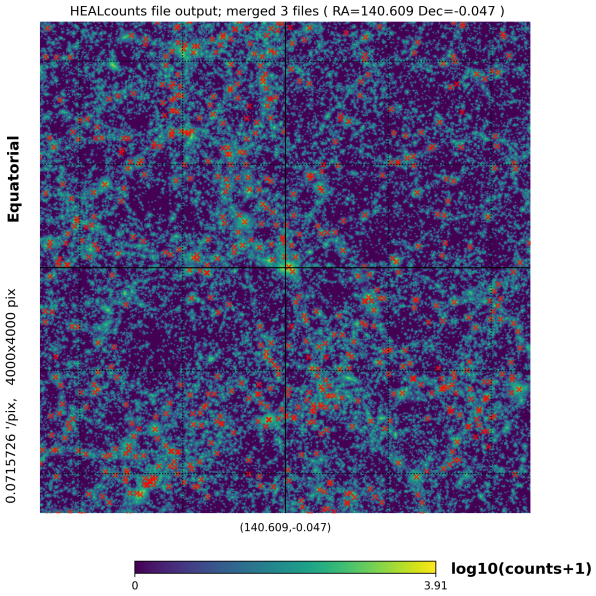


Figure 5: Output of the simulation upon superimposing three healpix files

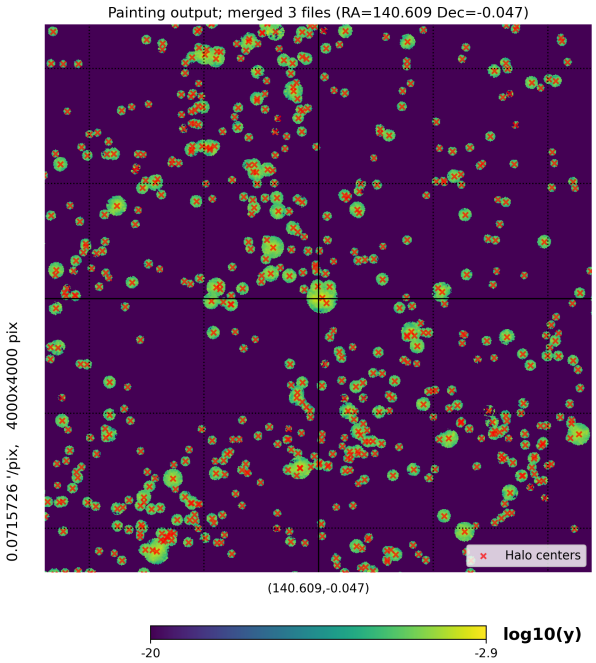


Figure 6: output of the painting mechanism with three healpix files superimposed

4. issue of the extension of the halo from the weighting mechanism

The same asymmetry can cause irregularities and gaps at edges of halos. Currently, zero-value pixels get a corresponding zero y-compton value in the painting mechanism. This poses a problem in the case that the GNFW predicted profile is predicted to have signal at a given distance away from the halo centre: but the healpix simulation output does not have signal there anymore. The overall GNFW is still preserved for the halos at any distance. Therefore, if there is any non zero pixels in that annulus of interest, they will get their signal boosted to preserve the signal. This creates an additional problem: namely: the GNFW profile is not supposed to follow the dark matter profile 1:1; that is the whole point of the masters thesis.

potential solution: for empty pixels: give them a flat weight: but non zero weight

problems: this will cause discontinuitites in the signal: the transition from the asymmetric signal to the symmetric signal at the boundary will not be continuous.

potential solution 2: use the information from the field particles. Apart from the halo particles; there are also field particle files: corresponding to particles filling up the space around the halos.

This is based upon the assumption (which also underlies the painting of the dark matter halos as well); that the asymmetry of the dark matter distribution is followed 1-1 by the gas: but just that the profile may be more extended

My notes: caviats in this approach: the GNFW profile assumes that the particles interact with the spherical dark matter halo underneath. It is an empirical model derived from simulations around dark matter halos specifically, and different physics describes the intergalactic medium than the outskirts of the halos.

implementation idea:

when querying the tree

MEETING NOTES:

1. Try to find if there is any information saved about the radius of the particles 2. get the comoving distance thing to work: to see which halos correspond to which particle files 3. Download the field particle files: paint using field particles 4. Think about painting asymmetrically: use the eigenvectors of ellipsoids as opposed to a spherical system 5. 2-halo terms 6. Get the physical profiles with radius and not with mass

AFTER MEETING NOTES:

Used to use halos-lightcones for halo information, but they don't have a lot of information (basically only N, and com position of the halo); However, most information is stored in the halo-timeslice files: they contain more information; just stored in a different slice. Then have the list: halo-timeslice-index: and it maps which halos in the timeslice sectoin belong to which halos in the lightcones: use it to map;

instead of calculating the virial radius using the cosmology:

Healpix workspace in Julia

Lab 20: Reading previous implementations on particle painting

Date: 11/02/2026

"Simulations of the microwave sky" Sehgal_2010

N-body simulations evolving only dark matter under gravity (like Abacussummit)

section 2.4 SZ signal:

2.4.1: massive halos at z < 3

The thermal Sunyaev–Zel'dovich (tSZ) signal is incorporated by post-processing dark matter halos identified in a gravity-only N-body simulation. Halos above a specified mass threshold are selected from a friends-of-friends (FoF) catalog. For each halo, the associated N-body particles are interpolated onto a non-periodic Cartesian mesh centered on the halo using a cloud-in-cell (CIC) assignment scheme, producing a continuous matter density field $\rho_m(\mathbf{x})$. The gravitational potential $\phi(\mathbf{x})$ is then computed by solving Poisson's equation,

$$\nabla^2\phi = 4\pi G\rho_m, \tag{6}$$

via a non-periodic fast Fourier transform (FFT). The location of the potential minimum defines the halo center. This procedure preserves the intrinsic triaxiality, concentration, and substructure of the simulated halo.

Assuming hydrostatic equilibrium, the gas satisfies

$$\nabla P_g = -\rho_g \nabla\phi, \tag{7}$$

together with a polytropic equation of state,

$$P_g \propto \rho_g^{\Gamma}, \tag{8}$$

where Γ is the polytropic index. The resulting gas density and pressure profiles are

$$\rho_g = \rho_{g,0}\psi^{1/(\Gamma-1)}, \quad P_g = P_{g,0}\psi^{\Gamma/(\Gamma-1)}, \tag{9}$$

where the dimensionless temperature variable ψ is defined as

$$\psi = 1 - \left(\frac{\Gamma-1}{\Gamma}\right)\left(\frac{\rho_{g,0}}{P_{g,0}}\right)(\phi - \phi_0). \tag{10}$$

The normalization constants $\rho_{g,0}$ and $P_{g,0}$ are calibrated to reproduce observed gas fractions and X-ray temperatures.

The tSZ effect is computed by integrating the electron pressure along the line of sight. Assuming the electrons trace the fully ionized gas, the Compton-y parameter is

$$y(\hat{\mathbf{n}}) = \frac{\sigma_T}{m_e c^2}\int P_e\, dl, \tag{11}$$

where σ_T is the Thomson cross-section and P_e is the electron pressure, proportional to the gas pressure P_g. The projected y-contribution from each halo is added to the sky map.

Baryonic mass conservation is enforced by requiring the total gas and stellar mass associated with each halo to equal the cosmic baryon fraction,

$$M_{\text{baryon}} = \left(\frac{\Omega_b}{\Omega_m}\right) M_{\text{halo}}. \tag{12}$$

Mesh cells are ranked by gravitational potential depth, and gas is assigned until the baryon budget is satisfied, yielding a physically motivated truncation radius. This approach produces triaxial, spatially resolved pressure distributions that generate realistic tSZ morphology while retaining the computational efficiency of dark matter-only simulations.

Preserves asymmetry and internal structure, at the cost of increased computational time

2.4.2 intracluster medium at z < 3: treatment of fields

For very small halos and the intracluster medium: hydrodynamic profiles; the Battaglia profile is no longer applicable.

Low-mass halos and the diffuse intergalactic medium (IGM) at $z < 3$ are modeled using N-body particles not associated with the massive halos treated via hydrostatic equilibrium. Although halos can be identified down to $M_{\rm FoF} \sim 6.82 \times 10^{12}\, h^{-1} M_\odot$, their limited particle counts prevent reliable application of a hydrostatic gas model. Instead, a prescription motivated by the virial theorem is adopted, enabling a unified treatment of both low-mass halos and unbound particles representing the IGM.

Under the virial theorem, the internal energy of a virialized structure is proportional to its gravitational potential energy. Gas within forming halos is assumed to be shock-heated, and its temperature is inferred from the dark matter velocity dispersion rather than directly from halo mass. For each N-body particle, the local matter density ρ_m and velocity dispersion σ_v are estimated using a fixed number of neighboring particles (typically 32). The effective gas pressure is then defined as

$$P_g = \frac{\Omega_b}{\Omega_m} \left[\frac{(\gamma - 1)\, \rho_m \sigma_v^2}{2 \mu m_H k_B} \right], \tag{13}$$

where $\gamma = 5/3$ is the adiabatic index and μ is the mean molecular weight for a fully ionized gas. The gas momentum density is assumed to trace that of the dark matter,

$$(\rho \mathbf{v})_g = \frac{\Omega_b}{\Omega_m} (\rho \mathbf{v})_m. \tag{14}$$

A temperature floor of $10^4\,\mathrm{K}$ is imposed to represent the photoionized IGM, and helium reionization is assumed to occur instantaneously at $z = 3$.

The thermal and kinetic SZ signals are computed by projecting the effective gas properties onto the sky. Each particle contributes according to its estimated pressure and momentum, with its angular extent determined by the solid angle subtended at the observer. This prevents nearby low-density particles from producing artificially large temperature fluctuations when mapped onto discrete pixels. Mass and momentum conservation are explicitly enforced.

This approach yields a smooth transition between concentrated halo gas and the diffuse IGM. While low-mass halos and the IGM contribute substantially to the total tSZ flux, their impact on the tSZ angular power spectrum is modest. In contrast, these components play a more significant role in the kSZ signal, particularly at small angular scales, where the bulk peculiar momentum of diffuse gas dominates. The kSZ signal is comparatively less sensitive to simulation resolution, as large-scale velocity fields are well captured by the N-body dynamics.

Incorporating this into my project

This is a much more complicated analysis, preserving more detail than the "xgpaint" like approach for

"The picasso gas model: Painting intracluster gas on gravity-only simulations K_ruzor___2024 "

This paper uses the same hydrodynamical approach for painting the particle halos as the paper above; drawing directly from their work. The nuance comes from training a machine to learn the gas properties from underlying particle properties.

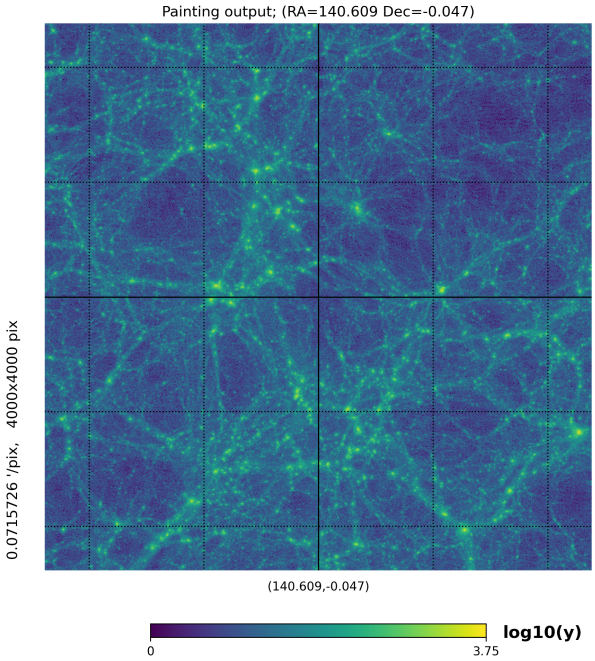


Figure 7: Including both field and halo particles into the healpix counts

Lab 21: Cleaning up code; painting field particles

Date: 13/02/2026

Completely restructured the code and cleaned it up.

Obtain the radii directly from the data rather than calculating them.

L2 vs COM in abacusfiles:

L2: iterative shrinking sphere: start with halo particles; compute COM; shrink the sphere around COM (to only keep like 90 percent of the particles); recompute COM; repeat until convergence

asymmetrical painting:

Include the field particles for painting

check the physicality of the painting with field particles:

pre-field particles; the y vs mass plot shows excellent agreement with the 5/3 theoretically predicted scaling:

1. Instead of painting on a sphere: paint on an oblate spheroid

'sigman$_e igenvecs_L 2com_u 16'$,' $sigman_e igenvecs_c om_u 16'$,' $sigmar_e igenvecs_L 2com_u 16'$,' $sigmar_e igenvecs_c om_u 16'$

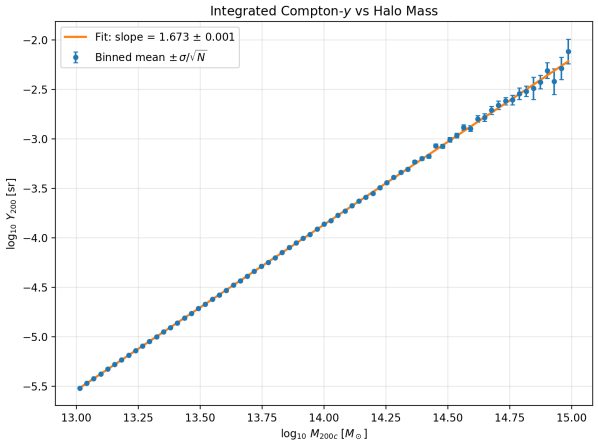


Figure 8: halo particles only

Lab 22: Allocating halo files to healpix files

Date: 16/02/2026

information from the healpix files headers:

have n+1 sublists: corresponding to the n steps which are superimposed for each healpix file

Boxsize: 1000 Mpc/ h: the size of the fundamental periodic cube;

the comoving distance (given as 'CoordinateDistanceHMpc'): on the order of 1500 Mpc /h : larger than the periodic cube: the lightcone extends further out: need to repeat the

Redshift given as well

for the code: need the comoving distance to the step values: the edges are the min and max from these files

the output from the file shows that the halo catalogs overlap with three healpix files: and but do not cover the full 3 healpix files

Halo shell edges for file LightCone0$_h alo_h eal - counts_S tep 0677 - 0682.asdf(Mpc/h) : 1352.3\check{}1382.0 Haloshelledges$
$counts_S tep 0671 - 0676.asdf(Mpc/h) : 1382.0\check{}1411.9 Haloshelledges for file LightCone0_h alo_h eal -$
$counts_S tep 0665 - 0670.asdf(Mpc/h) : 1411.9\check{}1441.5 Halocatalog comoving distance range(Mpc/h) :$
1364.7$\check{}$1451.0

for each halo in the halo catalog: in the "halo$_l ightcone"subsection we have in formation about the"Interpolated_C o$

for now:

1. use the alligned healpix map with the corresponding halo file 2. use only halos which correspond to the comoving range of the healpix files: perform a filter 3. paint to see if rigged features remain

solution for longer:

need to map the halos to the underlying pixels:

the halo files have a much higher range of redshift; and for each halo file; there is a corresponding healpix file which has a similar redshift: pair them up together: as this is most certainly the whole healpix file is in the halo file

for a sanity check: there is usually two healpix files which are fully in the halo catalog; and then there are two which are in between two halo catalogs;

pseudo-code for this data allocation: idea one

1. for all healpix files get the outer and inner edge: 2. for all healpix files create empty lists of halos properties;

3. for all halo files: get the outer and inner edge of the comoving distance; also get lists of comoving distances for each halo;

4. allocate the halos of matching comoving distances to matching empty healpix files: alongside with all of their properties

save as matching healpix to halo catalogs

pseudo-code 2:

1. for all healpix files get the outer and inner edge: 2. for all halos get the outer and inner edge:

3.assign the healpix files which fall whole into the halo files as matching files;

4. then for these allocations (usually two healpix files per halo file as a sanity check): go into the halo catalog and apply a filter for the halo comoving distances to match that of the healpix files;

5. the remaining outer and inner halos fall into the intermiedate files

the second one is faster: it does not require us to create new lists: however groups the two healpix files together per halo file:

Lab 23: parallelization crash course

Date: 16/02/2026

concurrent futures;

have classes of executors;

use ProcessPoolExecutor, as it assings separate global interpreter locks (GIL) in python inside a single process (in contrast with threadpoolexecutor: which even if u have multiple cores and nodes: python is only running in this one node still);

structure: define futures

futures = ex.submit()

process different tasks in the different threads;

then return the jobs:

for fut in as$_{c}ompleted(futures)(returninanyorderinwhichtheyfinish)$;

then msg.fut(result) return the results

https://docs.python.org/3/library/concurrent.futures.html

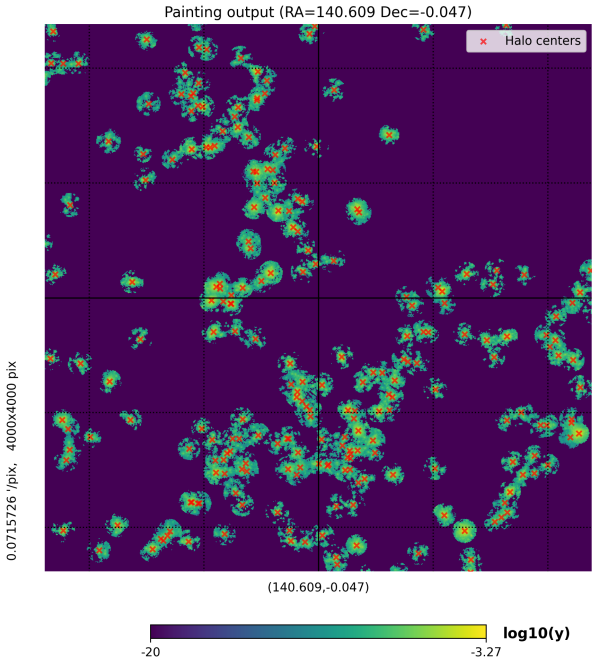


Figure 9: Caption

Lab 24: fixing memory bugs

Date: 17/02/2026

Spent like 10 hours trying to figure out why my code was not working: it keeps on killing my code due to memory issues.

I created a file with decorators: which times and counts the memory of my code in various places;

I also created a script which decreases the size of my nside for testing: in order to speed it up

for slurm jobs: go to icelake-himem: thats the high memory cpu

the new r90 is around 100 times larger than the 2 x theta$_2$00$ihavecreatedbefore: thisincreasesmemoryusagesub$
$asalloutputedarraysare$100$timeslonger$

this causes the code to crash for nside larger than 2048;

fixed this with an optimized weighting mechanism: and with an optimized painting mechanism

also created a vectorized fucntion for the weighting mechanism:

managed to create the simulation output for r search = 0.1 times r90; it is still 5 times larger than 2 times theta 200: (r90 is around 100 times larger than 2 r$_2$00)

upon all of this work in the past week to optimize the code to use less memory: it was also massively optimized speed wise: the whole painting output now takes 458 seconds on 8 cpus and 200 gb of memory:

this is using the healpy querying tree as opposed to kdtree

compute weights adds 100 seconds

The choice of r90 as the query radius introduces a systematic error.

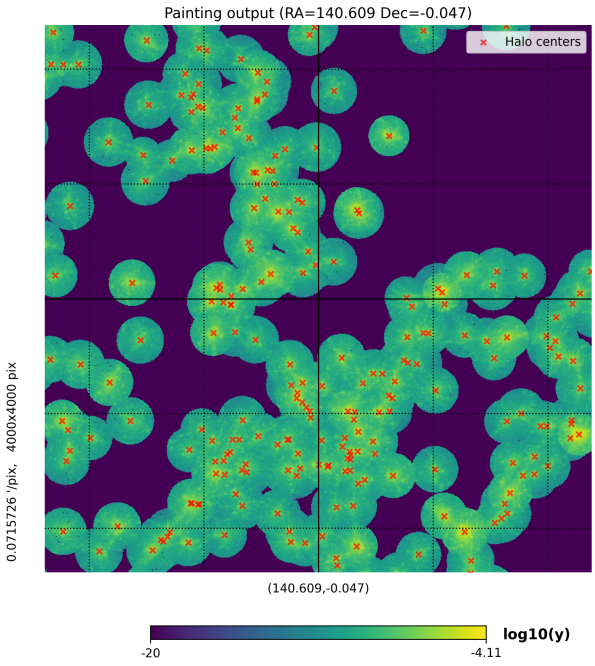


Figure 10: Caption

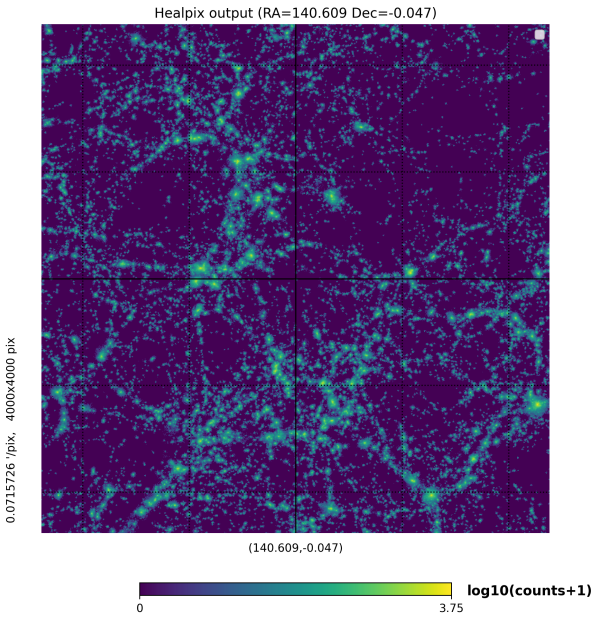


Figure 11: Caption

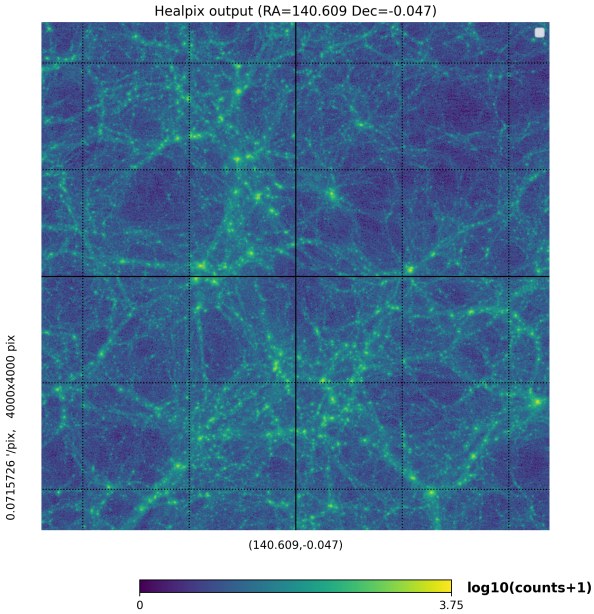


Figure 12: Caption

The overall signal to paint per halo is dependent on the mass of the halo and the distance from the halo centre through the battaglia profile.

Normalization is only per bins in the radial halos to preserve the correct asymmetry of the signal;

the overall signal then gets painted based on the total mass of the halo:

for large radii predict that the signal decreases as predicted by the battaglia profile;

NOTE TO SELF: for the field + halo files have a constant high value of particle counts at high radiuses: it should still drop down to zero which is not what is observed here with the current code: need to account for this

it is problematic to have large radiuses which overlap with the centers of other halos:

for particles: as we get double counting of the halos: this destroyes the asymmetric pattern: most visible for full field particles for large radii;

this was not a problem for xgpaint; or any codes which solely use the halo files and not the healpix files: we do expect overlap between halos to produce an increased signal; therefore this overlap convienently produces the asymetrical pattern for clusters of halos

however when we have the healpix information from the beginning: we are effecively using it in a double manner here; we are

For generation of the y map: taken for the halo mass

it gets redistributed within the query radius to match the total signal from a halo

the way in which r90 is defined is also not the best parameter

y map vs M accuracy for different search radii

:

Can find the optimal R by plotting how well the y vs M behaviour is conserved:

the closest the slope gets to 1.667 and all the halos follow the predicted behaviour the better

for field particles for very large radii; the signal from larger halos gets double counted: and the slope gets too high;

and aside:

systematic errors from using full field data

1. when using field data for clusters of halos close to each other: the halos with larger mass get a proportionally larger y mass signal:

this paints the whole r-search with a higher value: can see very distinct blobs on the screen

NOTE TO MYSELF: when plotting the y vs m: do we do that for the final map and take into account the asymmetry already: or do we do that for the allocations separately

2. general issue: when plotting the particle halos: the battaglia model which describes the dropdown is not a correct way of describing field particles: this then accumulates more signal for each halo than is expected;

potential solution: do the search radius

the way in which r90 is defined may not be the best parameter to use for search radii: i wanted to ask how is it derived:

does it draw from particle distributions? because if so: then the asymetry of the underlying halos and just like the different level of sparseness will make fluctuations in the radii distriubtions for the halos; potentially causing further deviations from the slope y vs m : 1.667

another systematic error:

because most signal is derived from large halos: we have created a mask larger than $10^13 Msun$: $sothatthesmallerhalosdontgetdetected$;

however this mask cannot be applied for the healpix maps (obviously);

there is multiple small halos surrounding the large halos; which will be absorbed into the signal of the larger halos: this is physically incorrect: the small halos by no means shohuld follow the battaglia profile set out for them

Lab 25: plotting y with distance away from the centre

Date: 19/02/2026

idea: can plot this on a single plot for different halos if we normalize the profile with R-200 for each halo;

this is supposed to follow the battaglia profile;

plotting y vs r / R200 can give us self similar results

1. need to create an array of r-200 for all halos then for the purpose of this plotting

2. need to then sample the profile at various r / r200 from r = 0 to r = search-r: (think if we can use our nsteps for this)

3. for each halo need to save the y parameter with r / r200 for these n steps

4. for the final plot then find means at these distances from different measurements

what we can plot on the same plot for the battaglia profile is

Lab 26: corrections after meeting

Date: 22/02/2026

1. The healpix inbuilt package shows improvement in calculation time: it does not require the action of building the tree; querying seems to be faster as well

2. correcting r-90-i16 to be the right size using abacus documentation:

3. large search radii are gonna inacuratelly treat the small halos (as it treats them as far away from the center things following the battaglia profile whereas it should treat them as their own thing*;

ideally want to decrease the search radius to be around the virial radius**; and then decrease the mass threshold for halos as much as possible to capture all of the halos: and make them follow the battaglia profile.

Have decreased the search radius to 1 times r98: where 98 percent of the dark matter particles are included: should

* for our large search radius: which right now is about 15 times r virial: plot the y compton signal vs r in two different times:

1. probe the y signal directly from the halos in the healpix tree: this will already include double counting of small halos and overlap for all halos (as query overlaps the signal by definition)

2. probe the y signal per halo form the final y-map (redo the search but on the final map)

Both measurement points are susceptible to the double-counting and overlap of the halos: as overlap is unavoidable for the painting mechanism which utilizes healpix maps. However what is different for these two codes is that the latter gets artificial boosting of asymetrric structures. This is not true: this would be true if the battaglia profile was not imposed radially: with our current code this is not true

the deviations in the 2026/02/22 and 2026/02/23 graphs can come from the binning changing: as always have fixed number of bins:

paint with use-weights disabled to check if that is what is causing the issue

** in order to choose the correct radius to be the search radius:

in **Battaglia_2012** they also note an additional problem (apart from double counting and non physicality of the profiles): the tsz signal can sometimes not converge with radius: and our cutoff becomes arbitrary; whatever we choose will influence the value of the y profile that we get;

NOTE TO SELF: still havent solved the problem of zero pixels not giving any signal for the y compton parameter: i do need to use the 2 halo term as opposed to only using the 1 halo term; : resolved by using the field particle maps as opposed to normal maps

The largest deviations between our analytic/pasted profile spectra and the full simulations are for the highest mass (M500 & 4.2 $\times$ 1014M) clusters, particularly on small angular scales. The deviations between the pasted profiles and the simulations in this mass range arise from the increased level of substructure and asphericity in massive clusters in comparison to smaller objects due to the more recent formation epoch of large systems in a hierarchical structure formation;

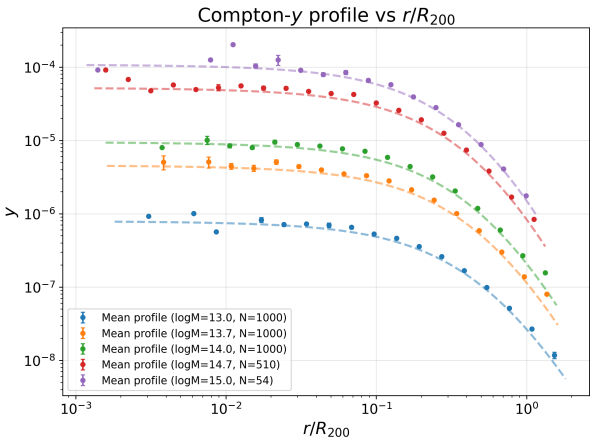


Figure 13: the y-map profile for halos

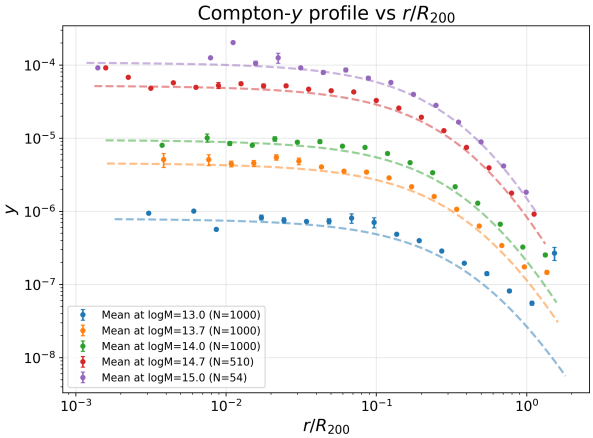


Figure 14: the y-map profile for halos after painting

Lab 27: average y-compton profile for analysis

Date: 22/02/2026

1. need to create a larger grid: vary the GNFW parameters themselves

1.1. Battaglia paper (2011):

Treat each parameter as a separable function of redshift and mass: evaluate it halo by halo; gives good results: with the fit parameters constrained to be of the following form:

$$A(M,z) = A_0(\frac{M}{10^{14}M_{sun}})^{\alpha_m}(1+z)^{\alpha_z} \tag{15}$$

where P0, Beta and xc are the parameters which we include as A;

the exponents on top of the mass and the z dependence should not be touched; this is because they come from self-similar scaling of gravity or redshift dependence: they do not hide any gas physics within them and therefore they do not represent free parameters

The amplitude prefactors: A0 etc are the things that hide the gas physics in them: should vary them

A lot of previous case works suggests keeping $\alpha = 1.0$ and $\gamma = -0.3$ as constants of given values: as it is hard to constrain them observationally;

the inner slope gamma probes the cluster core: radii well below R500: but the point spread function spreads everything out there;

The parameters are so degenerate with each other and other variables: that its not meaningfull to constrain them and compare different masses and redshift slices

We do not have halo by halo data: we have stacked profiles: our data limits us to what we can meaningfully vary in the bayesian analysis:

can vary an overall scaling factor in the P0, Beta and xc parameters

Also they perform the weighting for each mass range separately to see how the best fit varies with the different masses: i dont personally see a point in that;

->When they fit the GNFW profile they weight each radial bin by the internal variance of the cluster profiles with that bin; so that the outskirts and irregular bits contribute less to the fitted parameters;

->For later analysis of the power spectra if decide to cross-correlate with CMB lensing: need to watch out for the cutoff radius: as abrupt changes in real space will also cause some funky behaviour in fourier space:

QUESTION: is it better to do a 3d grid: and then create the external four parameters on top of that (as all halos for one map get the same compton parameters): or is it better to create a 7 dimension grid from the beginning)

1.2 xgpaint code

XGPaint code: created a break model: for sufficiently small halos they implement a break model: could this be because gas at smaller scales follows some other behaviour?:

my thoughts recently: i started thinking that the radial profile fits are too good to be true considering how many systematics we have with the overlaps etc

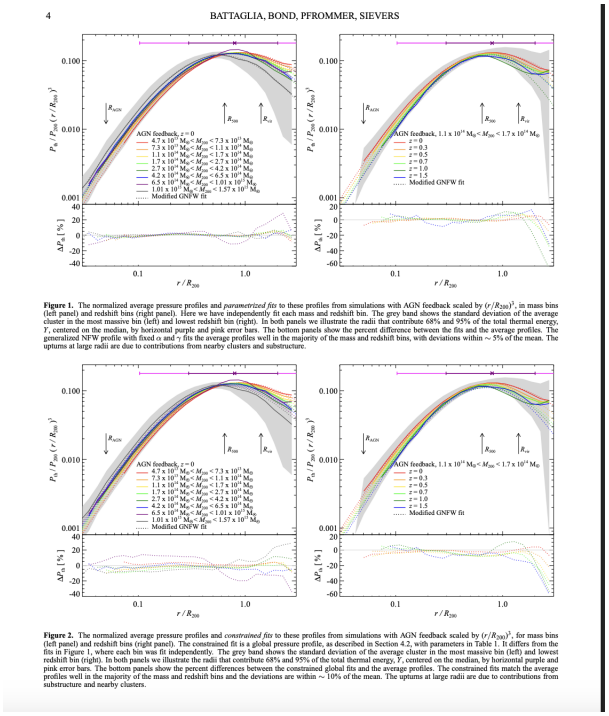


Figure 1. The normalized average pressure profiles and *parametrized fits* to these profiles from simulations with AGN feedback scaled by $(r/R_{200})^3$, in mass bins (left panel) and redshift bins (right panel). Here we have independently fit each mass and redshift bin. The grey band shows the standard deviation of the average cluster in the most massive bin (left) and lowest redshift bin (right). In both panels we illustrate the radii that contribute 68% and 95% of the total thermal energy, Y, centered on the median, by horizontal purple and pink error bars. The bottom panels show the percent difference between the fits and the average profiles. The generalized NFW profile with fixed α and γ fits the average profiles well in the majority of the mass and redshift bins, with deviations within $\sim$ 5% of the mean. The upturns at large radii are due to contributions from nearby clusters and substructure.

Figure 2. The normalized average pressure profiles and *constrained fits* to these profiles from simulations with AGN feedback scaled by $(r/R_{200})^3$, for mass bins (left panel) and redshift bins (right panel). The constrained fit is a global pressure profile, as described in Section 4.2, with parameters in Table 1. It differs from the fits in Figure 1, where each bin was fit independently. The grey band shows the standard deviation of the average cluster in the most massive bin (left) and lowest redshift bin (right). In both panels we illustrate the radii that contribute 68% and 95% of the total thermal energy, Y, centered on the median, by horizontal purple and pink error bars. The bottom panels show the percent differences between the constrained global fits and the average profiles. The constrained fits match the average profiles well in the majority of the mass and redshift bins and the deviations are within $\sim$ 10% of the mean. The upturns at large radii are due to contributions from substructure and nearby clusters.

Figure 15: idea for graph; from battaglia

i dont think the current normalization proceduce is correct, it reduces our information from particles files: so lets say i have a halo where at like a distance r have like a signal that is super high like higher than it should be if it was predicted by the battaglia profile: our code like makes sure that the weigthing for different pixels within this halo is alright but it doesnt acc account for the fact that this ring is higher (see some note before in this notebook): so in the final product it will just pint on the battaglia profile forcing everything to be following the profile: whereas we could utilize the information from the healpix not only shape but also magnitude

depending on what we want from the code

if we weant the anomalies of magnitude should do: i could do like a multiplicative factor: get the relative y parameter in each bin (as only proportional to N $^5/3$); $can i just do the 5/3 proportionality even though the$

get the the underlying y signal from the halo distribution from the interpolator: knowing the mass and radius away from the centre: then compare how the two profiles vary with each other: and change the painted signal to account for that

normalize relative to the whole halo: so the whole halo has to have the correct y-profile value and then normalize weights in bins as well

this will be more realistic: but the problem we will encounter is the increased contribution of absolute sparse territories

also for our analysis which will stack halos: wouldnt we actually average them out? like do we actually gain anything by doing this asymetry?

dealing with the two halo term:

for error handling: **popik2025impactshalomodelimplementations** recent paper;

Lab 28: more reading

Date: 24/02/2026

Citation which proves that gas inherits its asphericity from the asphericity of dark matter halos **2012ApJ...758...74B**; based on hydrodynamical simulations; (but the asphericity decreases relative to the underlying dark matter)

also found that the orientation of the halo is important: because tsz is a line of sight integral: this orientation influences the total y signal: can have differences at the scale of 10 percent!

in the output data have the orientation: eigenvectors and eigenvalues: could do a selection effect so if the halo is not alligned with us then we decrease its overall y profile value or increase it;

for such a large sample this will average out: shouldnt actually make a difference?

but then the intrinsic scatter associated with this can have an effect on the power spectrum;

calculating the eigenvalues and the corresponding shift in the signal will increase computational time a lot + also dont have one formula which describes how the projected signal chnages depending on the orientation

2. **Chiang2020SZTomography**

The halo mass function: describes the number density of dark matter halos as a function of mass

the power spectrum of the tSZ effect is the integral over this halo mass function weighted by the pressure profile squared

Lab 29: dealing with the 2 halo term; matching our simulated data set to the real data

Date: 25/02/2026 sample the final y map in random positions to get the average value of the 2

halo term: substract from the average values got from querying only around the halo centers

redshift: I have

1. **Schaan_2021** for matching data to the simulation outputs

III. Boss galaxy sample

IV. Analysis A. Filtering The CAP filter: (Compensated aperture photometric filter)

take the mean inside the disc of aperture range and then take the mean for a the annulus of theta to theta sqrt 2: that way the same size of the disc and the ring

the angular size is the same for all halos: its not halo dependent

why?: they use it to get rid of foregrounds; but it also stabilizes outskirt behaviour and removes constant offsets

do it to be consistent

Appendix B: Getting the 2 halo term for the samples using the cap filter:

The best thing about this masters project is that the simulations account for the two halo term without the need to having to create semi analytical prescriptions for it; like the need was in virtually all previous work:

tSZ Observable

The thermal Sunyaev–Zel'dovich (tSZ) effect arises from inverse Compton scattering of CMB photons by hot electrons. It is characterized by the Compton-y parameter:

$$y(\hat{n}) = \frac{\sigma_T}{m_e c^2} \int dl, P_e(l, \hat{n}), \tag{16}$$

where:

- σ_T is the Thomson cross-section,
- m_e is the electron mass,
- $P_e = n_e k_B T_e$ is the electron pressure,
- the integral is performed along the line of sight.

The observed CMB temperature fluctuation due to tSZ is proportional to $y(\hat{n})$:

$$\frac{\Delta T}{T}(\hat{n}) \propto y(\hat{n}). \tag{17}$$

Thus, observations produce a sky map of the Compton-y field.

Spherical Harmonic Decomposition

The observed y-field on the sky is expanded in spherical harmonics:

$$y(\hat{n}) = \sum_{\ell m} a_{\ell m} Y_{\ell m}(\hat{n}). \tag{18}$$

Angular Power Spectrum

The angular power spectrum is defined as

$$C_\ell^{yy} = \langle |a_{\ell m}|^2 \rangle, \tag{19}$$

which measures the variance of fluctuations as a function of angular scale.

Angular multipole ℓ corresponds approximately to angular scale $\theta \sim \pi/\ell$:

- small $\ell \to$ large angular scales,
- large $\ell \to$ small angular scales.

Relation to Correlation Function

The angular correlation function is given by

$$\xi(\theta) = \langle y(\hat{n}) y(\hat{n}') \rangle, \tag{20}$$

and is related to the angular power spectrum via

$$\xi(\theta) = \sum_\ell \frac{2\ell + 1}{4\pi} C_\ell^{yy} P_\ell(\cos\theta). \tag{21}$$

Halo Model Description

In the halo model, all matter (and gas) is assumed to reside in halos. The tSZ power spectrum can be decomposed as

$$C_\ell^{yy} = C_\ell^{1h} + C_\ell^{2h}, \tag{22}$$

where:

- C_ℓ^{1h} is the one-halo term (intra-halo contribution),
- C_ℓ^{2h} is the two-halo term (inter-halo contribution).

One-Halo Term

The one-halo term arises from individual halos and depends on their pressure profiles:

$$C_\ell^{1h} = \int dz, \frac{dV}{dz} \int dM, \frac{dn}{dM}(M, z), |y_\ell(M, z)|^2, \tag{23}$$

where:

- $\frac{dn}{dM}$ is the halo mass function,
- $\frac{dV}{dz}$ is the comoving volume element,
- $y_\ell(M, z)$ is the Fourier transform of the projected pressure profile of a halo.

Two-Halo Term

The two-halo term accounts for correlations between different halos:

$$C_\ell^{2h} = \int dz, \frac{dV}{dz}, P_{\rm lin}(k, z) \left[\int dM, \frac{dn}{dM}(M, z), b(M, z), y_\ell(M, z) \right]^2, \tag{24}$$

where:

- $b(M, z)$ is the halo bias,
- $P_{\rm lin}(k, z)$ is the linear matter power spectrum,
- $k \approx \ell/\chi(z)$ relates angular and spatial scales.

Bin	$\bar{z}$	$\bar{n}_\theta$	# of objects in bin (overall)	# of objects in bin (masked)
1	0.470	81.9	1,118,496	332,280
2	0.628	148.1	2,031,303	608,100
3	0.791	162.4	2,240,982	671,738
4	0.924	148.3	2,049,158	615,543

Figure 16: data

Physical Interpretation

- The one-halo term dominates at small angular scales and reflects internal halo structure (pressure profiles).
- The two-halo term dominates at large scales and reflects large-scale clustering of halos.

2. **Amodeo_2021** Follow up paper:

also raise the point of degeneracy of the y profile parameters:

There is significant degeneracy among all the GNFW parameters, so we fix two parameters sensitive to the profile properties at small radii, at the values suggested by previous cosmological simulations: t = 0.3 [54, 75] and xc,t = 0.497 × (M200/1014M

) 0.00865 × (1 + z) 0.731 [54].

estimating the 2 halo term:

$P_{tot}(r|M) = P_1h(r|M) + A_{t2h}P_{2h}(r|M)$

need to save both the: pre painting on the healpix data (so just halos and their painted y signals individually) and the after painting output again queried with the two halo term included: then substract these two stacked over multiple halos:

the connected github repository is https://github.com/samodeo/Mop-c-GT.git

which fits battaglia models to stacked data

3. **Liu_2025**

This is the data we will be using:

redshift range: around 0.4 to 1.0; have 4 bins:

mean redshift

mass range they used for simulations: log 12.4 to 3 log 14

mask the large mass halos

Pandey (2023): mass break model for masses smaller then break mass: choice of this break mass is motivated by simulations: which show a break in the self similar scaling of integrated y with mass roughly at this mass value: this is for correlations between y and the lensing parameters

Lab 30: Triaxial painting of halos

Date: 01/03/2026

in abacus loader:

1. extract eigenvectors and eigenvalues for each halo

eigenvalues, eigenvectors = ..

in pixel search:

2. query the disc to the largest eigenvalue of the halo: gather the pixels inside

hp.query$_d isc(search_r adius = np.sqrt(np.max(eigenvalues)))$

3. create a mask for the halos which are within the pixels of the ellipsoid:

to do this:

project the healpix vectors onto the basis of the eigenvectors of the halo;

for the pixels in each disc (upon first query):

in that basis the ellipse satisfies: (x/ np.sqrt(lambda x))$^2 + thesamefory + thesameforz <= 1$

where the lambdas are all N times the eigenvalues: as they also need to be queried to the extreme

and then give all the other pixels 0 value outside this mask: force this

4. for painting;

when painting the halo: should still paint based on the distance from the halo center: which doesn't change: so we paint the fields in radial patterns outwards for the halo fields: so we paint radially: and still preserve the profile

radii = np.sqrt(eigenvalues)

sigmar eigenvalues are not hard halo boundaries:

the eigenvalues are derived from the radius of momenta tensors:

based on the mass distibuted in different directions:

they encode the root-mean-square perpendicular size of the object relative to each axis: $I_i = M < r_\perp >^2$

question to boryana: do these already represent the real space eigenvalues;

i assume no:

when i evaluate the distance: i just simply substract the vectors from each other: which is fine on a cartesian grid but healpix is curved

literature review:

2024A&A...686A..97K: new traixiality formalism

Jing2002: the foundational paper: how to alter the battaglia profile for eliptical painting

obtain the shapes of the ellipses throught the scalings of the eigenvalues: the larger the eigenvalue the smaller the actual distance along this directions:

do scalings: do two ratios of b/a = sqrteigenvalue 1/ sqrteigenvalue 2 and c/a = sqrteigenvalue 1 / sqrteigenvalue3 which are just sqrtradius1/ sqrtradius2 etc: (the einverses of eigenvalues: the major axis will be the one with the smallest eigenvalue)

then the overall halo should satisfy abc = 4/3 pi $R_9 8^3 (thevolumeisconserved)$

then obtain a,b,c along their paths: (i.e solve these equations): remember that they are like the inverses of the eigenvalues: so perfect here for lengths

we have two sets of data:

from halo catalogs:

have the 3d halo profiles in cartesian coordinates: have the physical radii r98 and eigenvalues and eigenvectors:

what we want to conserve is the whole mass of the halo and the whole volume of the halo; when we do the projection in 2d

translate that to get the physical shape of the ellipses: define the principal axis (semi major; semi minor; semi midi) and their relative lengths;

need to project the ellipse on the healpix map:

then using this projected value: from the projected two eigenvalues of the ellipse we need to create an effective radius which we will use

so for right now: we calculate the y profile through actual 3 d integration and then when we query and paint we do two d: like before for the radial profile: but now we need to be more carful cause its an ellipsoid: so what we probaly want to do is calculate y the profile for the effective radius: meaning the radius which has the same volume as the underlying halo sphere: and that we established just above: (this is then simplying the LOS itegral to still only be a sphere but effective area sphere);

but we want to query and paint only for the projection which fits along our line of sight (because that is the information of our pixels like this is the limitation): so we need to recalculate this r effective for 2 dimensions when we query and do weights and stuff:(this should be trivial, it should pop up wehn we project the halo to

for the painting mechanism: in order to paint elliptically:

define the equivalent radius which will paint on the ellipse around:

this is for 3d

$\zeta^2 = \frac{x_1^2}{q_1^2} + \frac{{x_2}^2}{q_2^2} + x_3^2$

Lab 31: Triaxial halos: pseudocode

Date: 03/03/2026

1. get the dimensions of the ellipse: from conservation of volume get the a,b,c and their directions

2. project the ellipse onto the healpix map

projection onto the plane: https://math.stackexchange.com/questions/1814304/what-is-the-support-function-of-an-ellipse?

this is done using matrices: better in python than angle calculations as done in that paper i read a few days ago

define a tangent plane on the healpix map to the halo vector: approximate as cartesian:

-> two orthonormal vectors to the n0 vector of the halo (t1,t2)

write the 3d ellipse as a matrix;

-> Q = sum ei ei$^T/(ai)^2$

III. Project the Q matrix onto the tangent plane:

t1 Q t1$^T = S11; t2Qt2^T = S22; t1Qt2^T = S12;$

IV. Solve for the eigenvalues and eigenvectors of the second ellipsoid:

a$_p roj, c_p roj$

the distance from the ellipsoid centre can be drawn in a quadratic representation:

r**2 = S11 * u * u + 2.0 * S12 * u * v + S22 * v * v

where u and v are the positions of the pixel realtive to the halo center

V. get the distances of each pixel to the center

this is np.sqrt(a$_p rojtimesc_p roj) * r$

3. calculate the querying radius in 2d on the healpix:

-> when projecting the 3d halo onto the 2d healpix obtain the projected a' / c' now: will depend on the 3 previous eigenvalues;

-> then query using this new fake radius: the signal should be the same inside this radius: the edge of this radius is also derived from the previous edge (do up to 4 times this): also weight using this radius

projection onto the plane: https://math.stackexchange.com/questions/1814304/what-is-the-support-function-of-an-ellipse?

the general quadratic form taken from wikipedia which cites this: https://see.stanford.edu/materials/lsoeldsee263/symm.pdf

https://www.geometrictools.com/Documentation/PerspectiveProjectionEllipsoid.pdf

Including the line of sight integral

The current implementation only has spherical evaluation of the y compton signal as the LOS integral.

In order to fully implement the changes that the line of sight integral brings to the elliptical profile we need the initial 3d information from each halo as well as the 2 d information:

$$y_{\rm model}(x_\xi; l_p, p_i) = \left(2 l_p e_\parallel\right) \left(\frac{\sigma_T}{m_e c^2}\right) \int_{x_\xi}^{\infty} P_e(x_\zeta) \frac{x_\zeta}{\sqrt{x_\zeta^2 - x_\xi^2}}\, dx_\zeta \tag{25}$$

where the x squggly is the scaled version of the squiggly

$$\zeta^2 = \frac{x_1^2}{q_1^2} + \frac{x_2^2}{q_2^2} + x_3^2 \tag{26}$$

with q1 and q2 being ratios of eigenvalues of the halo

for a full analysis we would need the three eigenvalues and the direction towards us: and from that we could construct additional 4 dimensions of the grid for our y profile integral: then we evaluate the y profile using these 4 parameters: so we would do the integral in 3d and automatically that would project on two d: then on 2 d for this specific halo we apply the weigthing with respect ot the 2d effective radius and then we paint for this profile. but then the y grid will increase the computation time significantly: this will be very expensive and computationally heavy;

actually lets query up to largest eigenvalue and then cut out pixels

discussion on why to not do this exact treament

the particle information in the halos already preserves the asymmetry;

painting using the elliptical effective radius is therefore reduntant so is the LOS: we can just paint using the spherical battaglia: and only the extent of the halo we can get using the elliptical edges

Lab 32: Refining the code before the inference framework

Date: 04/03/2026

Halo mass range

between logM 11 and logM 11.5 there is 10 milion halos: for logM 11.5: 9 million halos; for log M 11: 19 million: increases the time from 30 minutes to around 52 minutes

inference framework

1. create multiple interpolators: around 150 interpolators: which will vary different battaglia profiles parameters:

then when evaluating maps; we evaluate the maps using the

HOD

HOD: halo occupation distribution: the probability that a given halo hosts galaxies in it: have center galaxies and satellite galaxies; from the halo catalog: put them on healpix maps and create HODS on them using a preexisting code; then have this one output per each healpix map (per each halo catalog): or multiple outputs where we vary the HOD parameters: the liu paper is concerned with LRG (luminous red galaxies): very massive galaxies, dominated by old stars

HOD abacus reading

Yuan_2021

bernoulli random variable for the central galaxy: if the mass is larger than the critical mass; poisson random variable for the satellite halos for all satellite halos;

Hydra

for running multiple files using SBATCH and for config files changing: can loop over the parameters in the slurm job

Lab 33: coding the mock data generation

Date: 06/03/2026

the outputs we need:

1. positions of galaxies for the redshift bins of bin 1 and bin 2

2. cumulative y-map in all ranges of z which contribute;

next steps:

I. code for convolving with gaussian beam

II. For bin i: place the galaxies on the y-map

extract the y values around each halo in an annulus around it: do CAP on it: repeat for 1 to 6 arcminutes:

CAP: mean inside disc r -

Lab 34: Introduction to nested sampling; GP emulators and samplers

Date: 09/03/2026

Reading:

Want to create a LHS (Latin hyperspace) grid of interpolator values

samples all dimensions uniformly: instead of clustering points at the edges

workflow:

create 125 interpolators

do simultaneous paininting of the maps using this 125 interpolators: for each redshift

for bin 2 this is from z = 0.5 to z = 0.75 roughly 20 maps we need to paint like this: so it will be 2250 maps

for bin 2 the maximum and minimum redshfits of the y-map files and the lrg files will be determined by the two extreme y-maps:

$\text{LightCone0}_{t}otal_{h}eal - counts_{S}tep0588 - 0593.asdfwithz = 0.745394271909432$

$\text{LightCone0}_{t}otal_{h}eal - counts_{S}tep0689 - 0695.asdfwithz = 0.503574707007947$

filter the halo files to accept only halos within this range

pseudocode: sum$_{ym}aps$

to save space: sum the y maps onto one of the redshift slices:

at each time can only hold 3 different redshift slices for the y maps: as each redshift slice takes up roughly 200 GB (1.5 GB times 125); and i only have space for 1TB on my rds: including also the halo catalogs simulation outputs this is the limit.

sum the pixel values on top of a selected redhsift slice now taken as the base

use blosc compression: which is good if there is lots of zero value pixels as then uses less memory for those pixels: within ASDF

https://github.com/asdf-format/asdf/issues/775

GP emulator from: https://scikit-learn.org/stable/modules/gaussian$_{p}rocess.html$

Lab 35: Binning problems

Date: 10/03/2026

The irregularity observed in the radial profile diagrams (pre superposition) are artefacts of the bin assignment when doing create$_{yv}s_{rp}rofile$: $thisneedstobesolvedfortheCAPaperture$: $asitinfluencesthemeasurement$

total storage on my rds: 1TB:

for halo files: 117 GB

for healpix files: 21 GB

for interpolators: 32 GB

for each slice:

need to compress the files using blosc compression: so have 1.4 GB instead of 3gb:

1.4 GB times 125 maps: 175 GB:

at first going to run 3 slices

also generate the lrgs for the same slices

idea to speed up interoplation:

all interpolators evaluate on the same poitns: we dont need to repeat computation:

Lab 36: HOD parameters

Date: 13/03/2026

Yuan_2021

Use the HOD package: which has an inbuilt variation of parameters which can be varied during the analysis:

redesigning the structure of the code to fit both the HOD and the battaglia parameters

inputs: 4 battaglia parameters; 9 HOD parameters to vary

for y maps: vary battaglia parameters: get 125 points on the lattice hypercube which produce 125 different summed y maps

for the galaxy catalogs: vary the HOD parameters: get 200 points on the lattice hypercube which produces 200 galaxy summaries

we then need to apply the 200 galaxy summarizes and the 125 different y maps to produce the final output: the 8 CAP filtered stacked maps in an npz file format

these 8 cap filtered maps are the final things we will then do nested sampling on:

exact gp training (like from scikit learn) scales like n^3 : $sowith125times200thatbecomesunfeasible$: ($times8aswellfortheCAPaperturepoints$!);

idea number 1. to check: assume that the physics of hod and the battaglia parameters is actually factorizable: then we can just perform the analysis separately for the battaglia parameters and the HOD parameters: train two separate emulators for the y maps and the HOD galaxy samples:

train the emulator for battaglia for a set of fixed parameters of the HOD: the standard ones:

$LRG_params : logM_cut : 13.1logM1 : 14.3sigma : 0.3alpha : 1.0kappa : 1.2alpha_c : 0alpha_s : 1s : 0s_v : 0s_p : 0s_r : 0Acent : -7.31189630e-01Asat : -2.41898268e-01Bcent : -9.37221616e-03Bsat : 3.74532572e-02ic : 0.97$

then train the hod parameters for a fixed fiducial y map:

the lrg only say which halos contribute: from the mass cut-off etc not how they contribute so maybe only a vertical shift

gp emulator pseudocode version one:

skitilearn gaussianprocessregressor

need to specifiy kernel: try the RBF;

works okay:

problems with GP emulators

nested sampling is sensititve to likelihood errors: need to ensure robustness of the emulator

badly constrained priors

:

have gotten my first results: show that cannot constrain gamma or alpha very well;

Lab 37: running the first dynesty script

Date: 14/03/2026

lhc sampling advantages: from mogp-emulator documentation:

samples more uniformly over the grid which we are intersted in: Latin Hypercube sampling ensures that each sample is drawn from a different part of the space for each parameter. For example, if four samples are drawn, then for each parameter, one sample is guaranteed to be drawn from each quartile of the distribution.

1. produce a gaussian process emulator from the lhc cube we have;

get a kernel: defined as the covariance between the function (which is the function we are interpolating over) value at two inputs;

https://peterroelants.github.io/posts/gaussian-process-kernels/ https://gaussianprocess.org/gpml/chapters/RW.

start with the squared- exponential kernel: popular for smooth functions; the kernel is specified using hyperparaemeters:

these hyperparameters get chosen using evidence maximisation: in order to find the most probable behaviour for our data

2. perform nested sampling on the emulator:

nested sampling introduction: https://www.imperial.ac.uk/media/imperial-college/research-centres-and-groups/astrophysics/public/icic/data-analysis-workshop/2016/NestedSampling$_J$*RP.pdf*

shrink prior space gradually; bettter for multimodal distriubtions

firs emulator; dynesty run

data information:

LRG: want AB: disabled

y-maps for the stacked y map:

641 -646 647 -652 653- 658 659-664 665-670 671-676 677-682

emulator: 125 points

dynesty: 500 n param

Moser_2021

diagram of 1 halo + 2 halo term

Lab 38: Planning the report

Date: 14/03/2026

robustness tests to include in appendices

:

1. discussion of the asymmetry skewness: boosting asymmetry by double painting

robustness tests to include in main text

:

1. discussion about the 1 halo vs 2 halo term: whether the painted profile is okay radially and in terms of mass

2. discussion about zoomed in images and allocating the halo centers

3. robustness of the emulator: check the emulator and then compare to a directly calculated stacked y map: leave one out analysis: repeat for all N points

Lab 39: Reading list

Date: 15/03/2026

https://arxiv.org/abs/1807.06209 : **liu2025fastbaryonicfieldpainting**

https://arxiv.org/abs/2401.13033 **coulton2025atacamacosmologytelescopesearch**

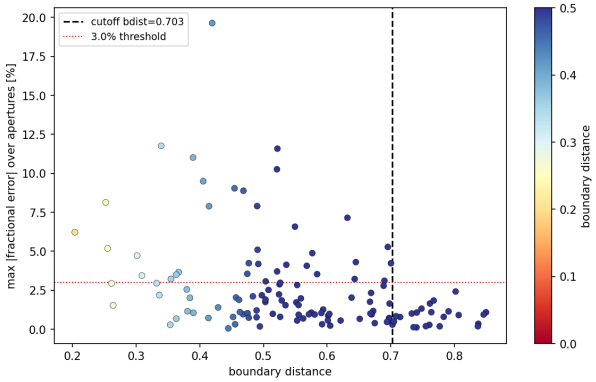


Figure 17

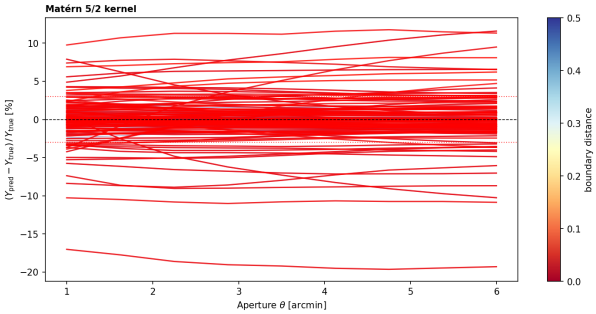


Figure 18

Lab 40: Testing the bayesian analysis framework

Date: 16/03/2026

1. GP emulator for the battaglia profile parameters has quite large errors: 3%; very expensive to run: multiple

N lhc used: 125 N param dimensions: 4

mostly problemativ at the edges: reaches uncertainty of 20%: however most values have errors higher than 1 percent

2. GP emulator for the hOD parameters: chose to vary only 3 for now: very small errors: all below 1%

N lhc = 125 N parameter = 3

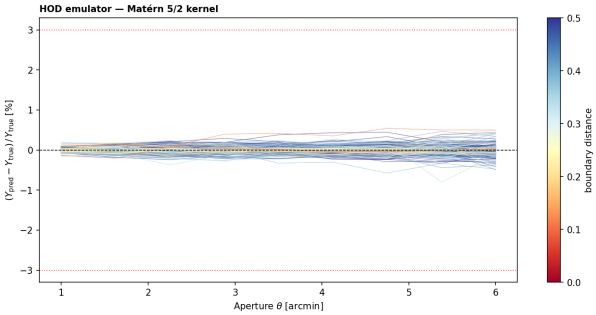


Figure 19

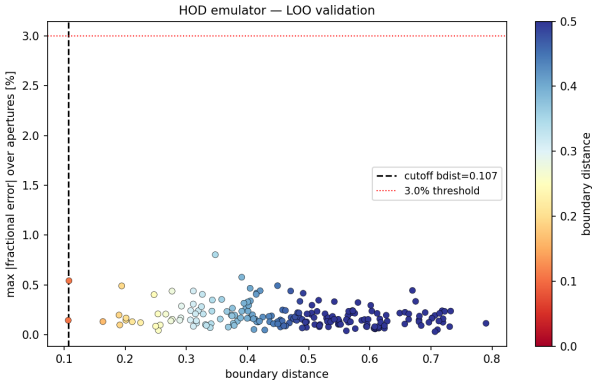


Figure 20

GNFW pressure model

P_0	Amplitude	[0.1, 30]	$2.0^{+2.0}_{-0.8}$
α_t	Intermediate slope	[0.1, 2]	$0.8^{+0.5}_{-0.2}$
β_t	Outer slope	[1, 10]	$2.6^{+1.0}_{-0.7}$
$A_{\rm 2h}$	Two-halo term amplitude	[0, 5]	$0.7^{+0.8}_{-0.4}$

Figure 21: Caption

need to see if this is because the emulator is undersampled: in order to check this: create a subsample of the lhc grid which will have roughly the same amount of points per dimension: so for 3d: 37 to equal to the 125 in 4 dimensions:

check the emulator errors then:

another factors to consider when the two emulators errors are not similar:

the HOD emulator is for

the required improvements for data-rerun

:

1. changing the priors:

upon discussions with boryana the priors chosen by me for first run:

logP0 = uniform [-2,2] alpha = [0.5, 1.5] gamma = [0.05, 0.8] beta0 = [3,6]

compare to priors of other people

alpha and gamma are degenerate: both probe the inside radius: + another issue is that the inside radius is acc smeared out by the gaussian beam;

Amodeo_2021: gave the estimates:

in all of their work they fix alpha at 1 and gamma at 0.3 which is based on hydrodynamical simulation runs done once: as they state that changing alpha and gamma did not change the estimates of the other parameters

however in their analysis: they vary alpha for a bit and they acc do get a meaningfull constraint: whereas in my analysis alpha and gamma are completely unconstrained unfortunately

they add the 2 halo term to constrain the 2 halo behaviour

Lab 41: varying hod parameters + initial results

Date: 28/03/2026

systematic: mass bias:

selection of halo masses is problematic; the true mass distribution of the data sample is not known and the tsz signal depends steeply on mass which makes the analysis tricky;

we could break the degeneracy with an external data mass calibration such as with weak lensing: however this is beyond the scope of this work (would love to continue with this in the summer holiday)

then we can instead argue what the liu 2025 paper argued:

choose to match the amount of gas measured at large aperture between simulations and the data (within the error bars of the data); ensuring that we are comparing galaxies with the same gas content: and therefore likely gas:

i have incorporated this into my bayesian inference analysis to better my fit: beforehand the code did not find anything useful

using fiducial vs deprojected maps:

i have chosen the data set from fig.4 of the Liu paper 2025: with the moment deprojected CIB (beta 1.6): as my forward model does not have any CIB: so the data

systematics: choosing HOD parameters: **popik2025impactshalomodelimplementations**

[for paper writing: cite this paragraph for explanations about what the forward modelling does: nice introduction]

Plan before next meeting: validate my outputs and explore the variations of parameters

1. validating the sims + stacking + inference:

sims validated using the painted profiles;

stacking + inference do one out analysis:

leave one stacked map out for a set of gnfw params: well inside emulator range

do the whole set up for the remaining 124 maps: and treat the one left out map as our mock data: the ground truth:

see how well the parameters are recovered

this will tell us whether the inference works: i.e if the current emulator is okay and the inference is reasonable

have spent 7 hours doing this because i corrupted my massive sim data and had to restore...

anyways:

1. idea number one:

simulate some error: 5% error + diag covariance;

now also motivated by the data vs simulation mismatch: our emulator mean is smaller then the data on average:

fix the inference to go through the last point: so for a given emulator predicted profile: fit the last point of the emulator profile to the last point of the predicted data: and then fit the shape only (so rescale the whole profile by this value):

the recovered parameters do not match the input parameters: they are way off:

the fit to the actual data points looks okay (CAP filtered data points): but the parameters are completely off:

Posterior summary: alpha: 1.0005 +0.2024 -0.2330 beta0: 4.7383 +0.5954 -0.7830 gamma: -0.6345 +0.0745 -0.0687 log10P0: -0.1980 +1.2263 -0.8465

True parameters (all$_s teps_s tacked$[15]) : $alpha$: 0.6272$beta$0 : 3.0263$gamma$: $-0.7007 log10P0$: -1.2385

this could be due to 1.) the profile being very flat and parameters being degenerate with each other: so acc a lot of possible minima and maxima which could fit; two: the anchoring method acc disturbing the structure of the inference;

The anchoring requires us to drop the last bin from the

have redone the same not using the anchoring and instead letting the parameters vary:

Posterior summary: alpha: 1.0190 +0.2106 -0.2200 beta0: 4.6391 +0.6058 -0.7033 gamma: -0.6090 +0.0880 -0.0773 log10P0: -0.4262 +0.6131 -0.5692

True parameters (all$_s teps_s tacked$[15]) : $alpha$: 0.6272$beta$0 : 3.0263$gamma$: $-0.7007 log10P0$: -1.2385

the three parameters apart from the P0 scaling do not change much fit the fit which is expected and good: the P0s main role is to scale the data points as opposed to change the shape of the profile significantly:

my main guess is that the 9 ACT points for a flat profile do not have enough constraining power to constrain the 4-dimensional setup: so for example the gamma and P0 are degenerate: then the alpha and beta also struggle with being detected as they control the innermost points of the data and these get smeared by the ACT beam inside;

next i added a covariance matrix to the data in order to simulate the errors a bit better: again run it anchored and not anchored

for the not anchored version:

Posterior summary: alpha: 1.0429 +0.1747 -0.2238 beta0: 4.7287 +0.3894 -0.7038 gamma: -0.5936 +0.0785 -0.0549 log10P0: -0.3108 +0.5578 -0.4031

True parameters (all$_s teps_s tacked$[15]) : $alpha$: 0.6272$beta$0 : 3.0263$gamma$: $-0.7007 log10P0$: -1.2385

for the anchored version:

Posterior summary: alpha: 1.0131 +0.1547 -0.2031 beta0: 4.8024 +0.4698 -0.7173 gamma: -0.6182 +0.0721 -0.0630 log10P0: -0.2505 +1.2932 -0.7200

True parameters (all$_s teps_s tacked$[15]) : $alpha$: 0.6272$beta$0 : 3.0263$gamma$: $-0.7007 log10P0$: -1.2385

did not change the fit significantly but did improve some of the erro bars: constrained it a bit more

also i set the mock data error to around 5%: but the fits get worse and worse and more degenerate on the corner plot; the higher we go in error bars: this is something to watch out for when it comes to act data which has a higher error

i realised that my beta0 parameter was acc outside of my prior range

1b: tried a different sample: specificially paying attention so that it will be further away from the boundaries of the emulator:

ranges of the prior: [5th percent to 95th percent of the emulator range] alpha [0.555, 1.446] beta0 [3.155, 5.834] gamma [-0.759, -0.089] log10P0 [-1.776, 1.774]

choose sample which is well within this range:

alpha: 0.9194 beta0: 4.0664 gamma: -0.5222 log10P0: -0.4847

number [52] in our $\text{all}_s tacked_p rofiles$

results for the anchored version of dynesty:

Posterior summary: alpha: 1.0342 +0.1673 -0.1557 beta0: 4.7524 +0.4772 -0.7272 gamma: -0.5232 +0.1234 -0.0833 log10P0: 0.0609 +0.8003 -0.8594

True parameters ($\text{all}_s teps_s tacked[52]$) : $alpha$: $0.9194 beta0$: $4.0664 gamma$: $-0.5222 log10P0$: -0.4847

results for the unanchored version of dynesty: alpha: 1.0195 +0.1711 -0.1628 beta0: 4.7477 +0.4739 -0.5506 gamma: -0.4844 +0.1264 -0.0991 log10P0: -0.2650 +0.4670 -0.6774

True parameters ($\text{all}_s teps_s tacked[52]$) : $alpha$: $0.9194 beta0$: $4.0664 gamma$: $-0.5222 log10P0$: -0.4847

much better fit but still see discrepancies: also the peaks are very very broadly distributed: basically cover the whole range of the emulator: when look at the diagrams:

==========

Varying alpha, fixing others to true values True value: 0.9194 0.9698 +0.0557 -0.0567

==========

Varying beta0, fixing others to true values True value: 4.0664 4.0397 +0.2378 -0.2583

==========

Varying gamma, fixing others to true values True value: -0.5222 Inferred: -0.5193 +0.0029 -0.0029

==========

Varying log10P0, fixing others to true values True value: -0.4847 -0.5012 +0.0140 -0.0159

this is basically within the error bounds: the

for the single run of the emulat

==========

Varying alpha, fixing others to true values True value: 0.9194 iter: 1409 | +500 | bound: 0 | nc: 1 | ncall: 8405 | eff(

==========

Varying beta0, fixing others to true values True value: 4.0664 iter: 1152 | +500 | bound: 0 | nc: 1 | ncall: 5885 | eff(

==========

Varying gamma, fixing others to true values True value: -0.5222 iter: 2760 | +500 | bound: 2 | nc: 1 | ncall: 38674 | eff(

==========

Varying log10P0, fixing others to true values True value: -0.4847 iter: 2799 | +500 | bound: 2 | nc: 1 | ncall: 38310 | eff(

also look into the parameters of nested sampling: seems like efficiency of beta is much higher than the efficiency of log10Po and gamma

2. exploring the emulated signals; varying only gamma; alpha; beta and P0: in order to check the degeneracies: and see what acc can be varied and seen:

firstly single parameter variation:

a) mock data well within emulator range:

the errors are very small; and dominated by the emulator itself not being able to predict the profile to a given accuracy: this is likely due to undersampling of the emulator for such a high prior range: which we will have to sample finer and think about the ranges: (the ranges are wrong because my emulator cannot fit all data points well: so need to extend them)

b) at the edge of the emulator range:

the errors still suprisingly small: also only dominated by the emulator error

2 parameter variation

b)

notes for Boryana about the asymmetry:

both the particle information and the information about the triaxiality of the halos; i am not sure it acc changes anything in the stacking: by logic if we average over multiple signals around galaxies this should all wash out: maybe only larger spread?:

this is also backed by a few papers: e.g **moser2023searchingsystematicsforwardmodeling**: where they explore diff los and traixiality: less than 1 percent difference

could do a test which checks this: run the whole sim for a fully spherical + weights off system to compare

also ask if boryana can think of any types of measurements where this asymmetry + traxiality could help for the future.

Lab 42: parallelizing the HOD parameter variation

Date: 30/03/2026

the end goal of this project is to produce inference plots varying both the HOD parameters and the GNFW parameters: as the HOD parameters influence the view of the GNFW parameters a lot: mass selection is important because tsz signal scales as $M^{5/3}$

for every 125 GNFW stacked y maps: we need to create stacked profiles: vary the hod parameters: around 200 realisations;

so the final data output is 200 files of all-steps-stacked: each row has 9 data points of the act 9 apertures (or 11 for ring-ring) and four points for the gnfw param variation

this will in effect be: 9 input points for the emulator (gnfw+hod) to 9 output points (9 aperture measurements)

XGBoost regressor or MLP neural network

at the meeting with Boryana we also discussed what would be the correct way to bring the forward model to the closest variation of the HOD parameters used in the ACT x DESI paper: so for the DESI LRG galaxies

NOTE TO SELF: need to do a lit review on how the HOD parameters influence the data;

Lab 43: doing corrections

Date: 06/04/2026

1. use more accurate representation of the HOD parameters in the ACT data then the default set by abacussummitHOD:

from this paper **yuan2023desionepercentsurveyexploring**: the same desi lrg data set as for our tsz paper;

our tsz pz2 sample is in the range around 0.5 to 0.75, whereas the parameters here are varied in the range either 0.4 to 0.6 or 0.6 to 0.8:

for an initial fiducial estimate: just set them to 0.6 to 0.8

parameters we will vary:

logMcut: logM1 sigma alpha k

to get the emulator range in this space: want to go from + 5 times sigma of the larger 0.4 to 0.6 range to - sigma of the other range: 0.6 to 0.8

Also, choose the columns which assume no assembly and secondary biases for now, as for the simplicity of the model, that's what I'm doing myself:

For gnfw, we use uniform priors bounded by the emulator range: the point of this master's thesis is to check whether the parameters differ from literature-predicted values, so we do not want to bias our analysis.

For the hod parameters, the desi paper used gaussian priors: because the flat priors were giving worse constraints, had a tendency to explore unphysical regions; but overall fit was very similar for most regions:

1.1 Creating an upper halo mass cut:

The tSZ signal increases as $M^{5/3}$.$Verymassivehalosdominatethesignal$; $ACTmaskst hosehalosfortszanalysis$:

Hilton_2021 in figure 7 describes the completeness relationship: number of clusters detected by the ACT divided by the number of clusters which actually exist in the catalog; they draw a halo distribution function from a tinker halo mass function: compute the tsz signal; apply act noise and enforce a certain SNR: then measure the completeness relationship

they find that with over 90% certainty they can detect clusters above 3.8 times $10^{14}M500c$: $wetakethis+anevenlydistributedparameterdeltainlogspace+-0.3tobethelimitandpriorar oundit$

the redhisft dependence over our area of interest: is very small: for now take constant

The tsz data analysis paper **Liu_2025**; they perfom masking on all ACT clusters which have an SNR higher than 6;

the tsz signal is constant with redshift, and the angular size of the clusters decreases as the redhsift increases: the detection acc gets easier with distance (maybe slightly counterintuitive) because the matched filter of the act works better for smaller clusters

UNITS: in the paper the result is given in M500c: need to convert it to M200m which fits the ABACUSHOD convention: the conversion in the **Hilton_2021** paper is done using this paper: **2013ApJ...766...32B** using the so called concentration-mass relation:

the definitions of the two masses: m 500 c and m 200 m are as follows:

M 500 c = 4 pi / 3 R 500 $c^3(500\rho c)$

M 200 m = 4 pi / 3 R 200 $m^3(200\rho m)$

in order to convert between these two types, we also need the relationship of how rho changes with the radius of the halo which is given by the NFW relationship;

Such changes are comfortably calculated using the colossus simulation code :)

2. Testing the true data from Henry Liu

also: i have rerun the inference using the good data from henry: and its not looking good: the emulator cant find any proper fits:

attach pictures:

its hitting prior edge corners + the profile does not match the data at all: there doesnt seem to be any fits in this emulator which ahve such a sharp increase of the profile at large radii; i think we may need to extend the grid?

i can see when I anchor the last data point this is especially clear: we hit the normalization factor upper bound and the beta lower bound: so the overall signal wants to get pushed up and the points at smaller radii want to get pushed down but the emulator doesnt allow for that

i also tested one thing: because we know that beta is one of the parametrs controlling the slope at higher radii: i constrained the prior to capture the shallower end of the emulator training range: and got results which still dont look good but look better hahah: i.e fit more points on the plot.

on the corner plot its still hitting edges though...

i had another idea: fix the Battaglia profiles: gamma and alpha to the predicted profile; and let beta vary; also anchor the last data point to ensure normalization:

this is all super dependent on the mass distribution of my halos though:

This brings me back to point 1 of my corrections: We need to constrain the HOD params correctly:

i have done what is in part 1 for my fiducial

3. fixing the ring-ring filter:

problem: first bin produces artificially boosted signals: its massive first vin and rest are small:

idea 1: its due to pixelation:

The central pixel leaks into the inner ring:

For the first aperture, the inner edge of the smaller ring could actually be smaller than the first pixel:

theta inner is 0.5 arcmin

for healpix of nside = 8192:

Npix = 12 N $side^2$:

area of one pixel: A = 4 pi / 12 $Nside^2$

size of one pixel sqrt of this = 0.43 arcmin; diagonal = 0.61

The galaxy positions are just positions on the sky: when superimposed onto my HEALPix map:

The galaxy can land anywhere in a pixel: so maximum distance from the pixel center for the nside = 8192 is like 0.3 (half the diagonal): for theta inner = 0.5 this shouldn't be a problem...

the code for my both the CAP and the ring ring pixels uses query disc: so it gets all the pixels whose CENTERS: lie within the required ring ranges; the less pixels oer nside the more pixelation: so that we have the pixels

2. The beam smoothing: ACT beam spread could cause spreading of the central high signal onto the first pixels: giving a higher signal in the first bin:

this would also affect the real act data though...

DUMB MISTAKE:

the good data i just got from henry saves only the ring ring filter apertures from 2.0 to 6.0 arcminutes!: so we can avoid this problem altogether for the dynesty inference thing

However, they do this ring-ring filter analysis in the figures 7,8,9 of their paper, and they don't have this massive peak in the first data point, but an interesting change of strategy: need to ask the author.

which makes me think

So they specifically avoid this region for some reason

Just plotted the ring ring without the first large peak, and it still flopped: there is still a decrease:

possible explanations: I explored how my mean mock data: averaged over all of my emulator points varies with the mock

calculated the mean value of my CAP and RING-RING apertures from the sims: and compared to the data:

CAP comparison: ap | $\mathrm{A}_disc|rawF_CAP|A * F_CAP(mock)|real(pz3)|ratio$ $-$ $-$ $-

my data also predicts the profile to be larger in small apertures: the same as for the cap filter. However, here areas increase only roughly linearly with space: so the increase in area causes the same problems, but now doesn't stop the decrease.

i have reread the paper to double check my logic **Liu_2025**. there is an additional component of the data treatment which we have not done:

The NILC decomposition: based on the paper **PhysRevD.109.063530**:

Lab 44: Planning the HOD LHC convolution

Date: 11/04/2026

1. input data: have 125 y maps:

2. wanted output: want for each of the y maps: stack them in 200 different ways and save on a 9-dimensional grid:

(stacked y profile)i for GNFW param variation (4d) + HOD (5d variation)

to make calculations more time-efficient:

1. for each of the 125 maps:

Firstly, preconvolve the map: cause we need to do that only once per y map: maybe create a script which will preconvolve all 125 maps and save them in a separate folder: called step 0677-0682 preconvolved:

because the files are large just do that one by one and save it in /home/ab2927/rds/hpc-work/tSZPaint$_{d}ata/preconvolved_{S}tep0677 - 0682$

Then, for each of the 200 parameters: run the stacking for the 125 maps but already preconvolved (so that among different 200 param runs we dont have to repeat this convolution)

for each HOD LHC parameter point; end up with an (125, 4,9) for CAP and (125, 4, 11) for ring ring: also write the names of the HOD params in this: so we know which ones it was

then repeat this for the 200 params: this can be run simultaneously actually: for all 200

then save this in like one directory: this is all small functions:

have a function/ script which at the end gathers this data and merges it into (20 thousand something, 9, 9)

then do the same for the 125 maps: so end up with 125 files each with

what to change in the current code to allow this change:

1. need to separate th convolution from the act data match.py –> run stacking.py pipeline and create space on my rds to fit an additional 125 x 3 gb of maps;

2. need to make the pipeline be able to loop over 200 indexes from the lhc hod text file and find corresponding run$_{h}oddatasetsforallthepointonthelhcfromthedatafiles$

2* need to create these 200 files from the run$_{h}od$: $alterthecurrentconfigtoallowustorunoverthedifferentparam$

memory problems: i created the convolved maps: this requires another 3 x 125 gb but also all the hod mocks get another 335 gb: this is not feasible to do with 1 T of data:

well it is in the sense that i could batch and do the operations in batches: so create convolved maps like 25 of them: perform stacking: then delete the maps: then perform stacking again blah: as the end products are cheaper: painful though cause have to delete large files: and everytime i want to run the stacking procedure: which will be more then once i have to redo this delete + batch etc

unfortunately my idea of saving up computational time will not work due to the increased volume of data i have: i need to instead convolve the data inside the slurm script and just roll with it; shame

after painful hours trying this to work: it doesnt: i think i cant run the 200 jobs at once: instead maybe create the convoluted y maps separately: and create like 10 hod files at once from the lhc: then run then delete the files.

idea number 2:

instead of plugging out all of the outputs from the lhc hod: which is 440 GB for the 200 hod variations in our emulator (huge) onto the rds: we could potetnially run the lhc hod inside each call;

then i would have space to save the 125 convolved maps: which would mean that i dont have to convolve the maps multiple times for each hod revolution

we could run the lhc hod files inside the code for each evaluation:

this is what i did. the convolution of the 125 y maps takes like 2.5 hours in total when run on one node (76 cpus). then the creation of the

Lab 45: further pipeline struggles + NILC analysis

Date: 12/04/2026

1. NILC:

Needlet decomposition: need to match the output of this to my mock data

2. running the pipeline:

i have succesfully implemented this new version of the code, hooray! however a new problem arises.. as e vary the hod parameters some of the lhv points end up creating massive galaxy catalogs (if m cut is small for example); this can reach up to 13 million halos; this blows up my query point radius ball: which increases the computational time of these nodes like 3 times

for now i have created a cap on the number of galaxies which we analyze to be 4 million (as 3 million was the fiducial run with standard desi lrg hod parameters which we chose): the code then subsamples the galaxies and picks random ones to analyze; as the number of galaxies does not influence the signal mean and properties: it serves to decrease the standard deviation

i would logically argue that this is better because it will also make the error in our emulator be of similar size in all bins: which will not bias the emulator to put different weights to different data points

also the actual paper has even less galaxies into the mean:

n z^- n^- of objects in bin (overall) of objects in bin (masked) 1 0.470 81.9 1,118,496 332,280 2 0.628 148.1 2,031,303 608,100 3 0.791 162.4 2,240,982 671,738 4 0.924 148.3 2,049,158 615,543

which is still so much smaller: this is mostly because they have limited view on the y map: only where desi sees it.

i have run it: after an hour and 5 minutes the code for 4 million sub cap: did 18 maps: so i am expecting the ones which need subsamping to finish around the 10 hour cap i have set for them; whereas ones which have like 3 million halos in the whole catalog: will finish in the

3. choosing the area of the healpix maps:

i should also mask the area of the healpix maps to match the same area used by the desi lrg x act data; and allow only for galaxies in this range to be explored: then we would see if this is correct

if this shows less galaxies then the true data: then decide the trade off: would we rather keep more galaxies in our mocks in wrong regions which do not have the ACT noise that well simulated or would we also like to keep less halos but in the physical region

also idea: when we do the are of act mask: and we count all the galaxies which are predicted to lie in a given region of desi x act: we could also get a validation of the sim parameters: because the number is roughly correct for the analysis

4. emulator for the 9 d

Lab 46: Goals to obtain in this masters thesis

Date: 06/04/2026

Ideally (but ambitiously), I would have all of this done before the master's thesis ends:

1. HOD variation as well as GNFW variation: this is a must

2. rerun my sims (as I am not sure if I attached one file in the stack + I think I should go further down with my data set) + also run for the act data set 1: ask boryana cause we could optimise this as the ranges for pz1 and pz2 are cross-aligning): do this after having the whole set-up for inference, working on my current data set

3. Run this on HydroSims to compare: Currently looking for hydro-sims with machine learning, which would have more massive halos

4. Maybe run my code with asymmetry vs without asymmetry to see if adding particle information + ellipticity actually changes results that much

Lab 47: literature review + new thoughts

Date: 15/04/2026

popik2025impactshalomodelimplementations discusses the importance of the choice of a forward model used for tsz measurements

advantage of using simulations in place of semianalytic modles: we can connect the real data directly to the implementation of astrophysics in simulations

this study investigates the importance of choosing the HOD parameters;

they discuss miscentering:

in a typical halo: the galaxies are placed directly in the center of a halo: and therefore aligned with the pressure profile of that halo: whilst satellite galaxies are equally distributed from the halo center: there is an underlying assumption that halo centers are properly identified and aligned with the galaxies within them:

if this is not true: then the cross-correlation signal would be dampened: for mass averaged profiles: manifest as a decrease of signal at lower scales: with a boost at some scale corresponding to description of the misceneters galaxies:

this has shown to produce sig offsets between forward models and observations: they model that here by doing

also modelling the mass distribution of a sample of galaxies is very important: this is because the tsz is very

Moser__2021: explored the impotance of modelling the galaxy sample: such as different sample selection in the simulation

moser2023searchingsystematicsforwardmodeling: minimal changes when looking at how the projection of 3D pressure profile chages the signals

Battaglia__2012 in this paper battaglia derives his profile and provides fits of the params based on hydrodynamical simulations;

usin hydro sims: calibrate the GNFW profile and set its specific coefficients to a given value;

measure the pressure profile in the sims: 1. find clusters in the sims (using friends of friends): this gives halo mass; cluster centers + cluster radii 2. around each halo calculate the p(r): by querying round and just measuring from the sims; 3. then also normalize the halos such that we can compare this across redshift + mass ranges 4. they fit the GNFW profile params on this: using just some regression algorithm

even though the params should be independent on redshift because of how we have constructed the self similarity: this is not true: and params shift with redshift all beta xc and P0 are varied: and all assummed to have a variation with z and m

whereas for our variation of alpha and gamma: we have treated them as free params (and assumed no variation with z which i am not sure about)

alpha gamma fixed: also see nagai 2007:

Nagai2007ICM argues that because gas behaviour is still mostly dominated by the graviational behaviour (dominated by dark matter component): therefore we can model the gas as the GNFW profile

the foundational paper of the field: this paper proves that the pressure profiles are self similar and we can therefore even establish the gnfw in this context

fit the params using observations of chandra and simulations of chandra: x-ray

Arnaud_2010:

2. changing the emulator ranges:

when looking at the battaglia paper for the pressure profile which is the framework for our work:

$$P_e(x; z, M_h) = X_e \frac{\Omega_b}{\Omega_m} P_{200,c}(z, M_h) P_0 \left[\frac{x}{x_c}\right]^{\gamma} \left[1 + \left(\frac{x}{x_c}\right)^{\alpha}\right]^{-\beta} \tag{27}$$

$$A(z, M_h) = A_0^x \left[\frac{M_{200,c}(z, M_h)}{10^{14} M_\odot}\right]^{\alpha_m^x} [1+z]^{\alpha_z^x} \tag{28}$$

scaling of the overall amplitude:

$$P_{200,c}(z, M_h) = \frac{G\, M_{200,c}(z, M_h)\, [200\, \rho_c(z)]}{2\, R_{200,c}(z, M_h)} \tag{29}$$

people have gotten beta = 4.01 recently like quite low values: i think the range of the emulator with only 3 is not enough: we should think more about which range to choose from for the profile

Date: 16/04/2026 note for myself so that i dont forget:

1. in the masters thesis talk about why we chose real space over fourier transforms 2. talk about the fundamental limitations of the gNFW profile

galaxy number discussion

also we should explore more the number of galaxies at which we are downsampling the galaxy query for stacking: right now its 4 million based on the fact that this number is similar to the number of galaxies which have been generated for our collection of halo catalogs spanning this required redshift:

this also ensures that the error of the emulator in most samples of the hod will be the same (if they reach the threshold of 4.000.00), whilst keeping the computational time feasible. however for samples which have not yet reached this threshold of 4 million: the error would be larger.

also quantify the error: see at what desired number of halos the error reaches a value which we are satisfied in: maybe that will be lower then 4000000 and we can reduce compute time

fixing ring-ring

i have also found the bug of my ring-ring filter:

it does not lay in the linear vs quadratic increase of are as i originally thought: it is because one figure in the appendix used a different definition of the ring ring filter:

it kept the inner radius theta 0 to be constant and 1 arcminute as opposed to varying it with respect to the outer radius and keeping the widht fixed. so now the ring ring filter still maintains some cumulative pattern: it is not purely sampling only 1 bin per data

checking mean redshift

for the generated halos from my files we are aiming for obtaining a mean redhisft of 6.23 for the final run so that data is consistent:

i had a query to Boryana about it because i realised i was not covering the correct range: The first and probably most important issue is that I realised I have covered a smaller redshift range for the y-maps in bin 2 than I originally thought. At the moment they only go from z = 0.5 up to z = 0.7. I think I mistakenly confused where the upper edge of bin 2 actually sits. Looking again at the diagram (below), the number of halos around z = 0.7 is still very high, so missing the 0.70.8 range likely skews the results. Do I understand correctly that this skews the signal magnitude because we are missing a population of halos (?) or does this get averaged out as we do the stack. And more importantly if i understand correctly it could also change the shape of the stacked profile. Since we are stacking with a fixed angular aperture, the physical scale that aperture probes depends on the angular diameter distance, which changes with redshift. So at higher z the same aperture probes different physical regions of the halos

upon discussion we decided to check the mean redshift in the halo catalogs:

Step0617-0622: z=0.6624, n$_p articles = 24,402,427,976 Step 0623-0627 : z = 0.6484, n_p articles = 19,139,634,343 Step 0628-0634 : z = 0.6339, n_p articles = 24,117,202,504 Step 0635-0640 : z = 0.6188, n_p articles = 19,981,036,861 Step 0641-0646 : z = 0.6049, n_p articles = 19,283,195,700 Step 0647-0652 : z = 0.5910, n_p articles = 18,678,580,322 Step 0653-0658 : z = 0.5773, n_p articles = 18,138,857,809 Step 0659-0664 : z = 0.5638, n_p articles = 16,731,068,683 Step 0665-0670 : z = 0.5505, n_p articles = 16,538,650,518 Step 0671-0676 : z = 0.5370, n_p articles = 16,205,756,453 Step 0677-0682 : z = 0.5236, n_p articles = 15,348,247,910$

Total particles: 208,564,659,079 Mean redshift (particle-weighted): 0.597816

small optimization for run stacking

the code asks for 32 cpu cores: for each asdf y map it gives it 4 nproc: so parallelization of 8 per y map for the healpix querying

Date: 19/04/2026 My thoughts about the emulator:

because it doesnt recover the points properly for 4 parameter variation: i think it does not reproduce the data well hence beginning to do any analysis with it is inherently faulty: also i need

did some tests on the loo of the emulator: and for points with very small values of P0 the relative errors were huge

also the emulator spans $3^{10^8} magnitude : ithinkthismaybetheculprit : icantimaginetheemulator findinggoodfit$
$withverylittlepoints : thelHChas125points :$

but this could work if the parameters where close to each other

Date: 22/04/2026 I re-performed the emulator test based on my doubts from three days ago:

It actually doesnt look bad:

the emulator inside the prior range: from 0.05 to 0.95 range of the emulator: has a mean error of 3.22 %, and a median error of 1.92% it is mostly the points close to the 5th to 95th oundaries which generate very high errors:

even within this prior range the only scary points are these:

14, bdist=0.1162: 32.12% 40, bdist=0.0515: 21.28% 89, bdist=0.0539: 17.68% 11, bdist=0.0566: 14.39% 109, bdist=0.0806: 13.74% 57, bdist=0.1497: 11.96

the point 14 and 57 require most attention as they are well within prior bounds:

point 14 corresponds to

alpha = 0.61876568 beta0 = 4.4236068 gamma = -0.44446857 log10P0 = 1.2970397

which are actually quite standard parameters so this is not good

my guess is that is it because of its high distance from any other training point of the emulator: it is 0.338; so not a lot to extrapolate from

point 57 corresponds to: alpha = 0.9636522 beta0 = 5.491644 gamma = -0.68742047 log10P0 = -0.5359804

the test then to perform and see how accurate is the dynesty: and whether us hitting the corners is the edges of the emulator is limited by gnfw degeneracies or is it caused by our data actually wanting to explore lower eta or different gamma we could do two things:

firstly input some randomly generated point in dynesty which is like within a comfortable middle range of the prior: top 70% : and run dynesty for like 5 variations 5 random points

also there is only like 83 points which fit within the prior range out of the initial 125: we cant get rid of this loss because we do need to generate points more further out then our prior but this is something to think about if we rerun the code

if this is recovered all good

secondly choose points close to the emulator boundaries: and see how our data behaves

it is recovered most times yay!

initially i chose a small error: around 5% percentage error: which is smaller then the around 15% error in the ACT data:

these are the plots

then i increased the error to be 15% these are the plots

nowhere inside the emulator does the data diverge: the only query is that we have much less constraining power

Lab 51: Checking new emulator range

Date: 04/05/2026

to summarize: my message to boryana and her answer:

Hi, I also did / repeated some preliminary tests on both the emulator and the inference pipeline.

First, I repeated the leave-one-out validation for the ring–ring filter in a more systematic way (see plots below). I defined the minimal boundary distance as the smallest distance across all four dimensions of a training point from the edges of the Latin hypercube normalised to the full parameter range (so between 0 and 1).

I then calculated the errors in the range of the inference prior (which i chose as 5%–95% of the emulator range, to avoid boundary effects), I find a mean error of 3.22% and a median error of 1.92

There are a few higher-error points within this region though. Most are in the 5–7% range, but there is one clear outlier with an error of about 32%. This worried me because it lies roughly in the middle of the training range. I think the most likely explanation is that it is relatively isolated in parameter space: its nearest neighbour has a minimal distance of about 0.338 (in units of the full parameter range), compared to a mean nearest-neighbour distance of about 0.23 across the data points in the training set. If we rerun the analysis, I can look into improving the LHC design (I just used a standard Python package) to see if there is more options on how to optimize training point choices.

[7:04 PM]I then tested how well Dynesty reproduces known inputs using mock data. I selected random points from the LHC, generated mock data using the emulator. This is similar to what I did for the 1,2,4 D i showed you once in the presentation, but now I also added realistic noise: the covariance matrix with the same structure as the ACT data, assuming around 15% fractional errors also similar to the data. I run this for around 10 test points, and dynesty does not drift toward the edges of parameter space (unlike what we see with the real ACT data) (see below). However, the posteriors are very broad, which is not ideal. It is consistent though with what we saw previously in ACT-anchored fits which didnt drift to the edges of parameter spaces, so I reckon this is the best we are gonna get using such high error bars? I was wondering, if we did pz1 as well as pz2 maybe that could help constrain it a bit more.

Then for fun and as a cross-test I also did the same tests but with data errors of the order of 5% as opposed to 15%, which looks much nicer as expected. Again dynesty never drifted: most times it recovers the true parameters, sometimes it misses slightly but never hits a corner. The ring ring filter has much better results then the cap filter, but if i understand this is to be expected because of the diagonal covariance matrix?

I think this points slightly in the direction that the true data is just not consistent with the current parameter range. for every dynesty I ran in the last couple of days with the mock data i always got fits which showed basically perfect alignment with the data on all apertures; whereas dynesty never finds a fit for the smallest apertures for real ACT data. I will think of additional ways to test this after exams, but this seemed quite reasonable to me? let me know what you think!

Nice job! Yes, I agree with all your conclusions – it really does seem like our current parameter range is unable to fit the real data – perhaps our best bet is to try a cheap test at first just to see whether any parameter range can actually fit the data – hopefully the answer is yes (and it really should be since GNFW has too many free parameters to describe a simple pressure profile) – it would be good to test this cheaply – one idea might be to not paint anything but just use the analytical result for the 1-halo term (i.e. the interpolator prior to putting it on the

simulation particles) with a frozen 2-halo term (you could just do something like total signal minus 1-halo and then freeze that as a function of M and z – not sure what's easiest!!

plan of action for today:

ideas:

-> vary different gnfw parameters for the $y_p rofile_v alues : fortheintegralitselfasopposedtotheinterpolator : andperformthisintegralforeachparametervariation$

-> don't use the healpix maps;

-> use the same lrg catalog as for the sims data

-> for each of the bins perform the integral;

-> add the beam convolution

-> weight by the number of halos in each bin; for each redshift bin i chose 150 mass bins

-> then on this averaged profile perform the semi-cumulative ring-ring stacking;

then need to know also simulate the two halo term which usually is taken care off

choose one full simulation output for the stacked profiles: one quite close to battaglia params (so that its reasonable)

then for the same set of gnfw params calculate the analytical version

substract those two to obtain the 2 halo term: assume this is constant across the parameter variation: apply this addition to all the analytical versions; but scaled by the $10^{logP0fiducial-logP0point} topreservetheove$

the results are wrong:

1) no parameters fit the data properly : which is not feasible: as i increased the parameter range much above the previous simulation-based lhc cube

2) i run a code to check whether the 1 halo analytical + 2 halo scaled term reproduced the simulated profiles for the same parameter ranges: which showed major flaws: there is differences up to 100 % and they shift with aperture ....

brain storm:

I. check number one: lets see if the 1 halo term analytically is actually okay compared to the sim based version:

-> this can be checked through: running this analytical prescription but not adding the two halo term;

-> then comparing all the points from the sim based lhc to this new analytical prescription: but only the 3 inner most points (2,2.5,3 arcmins): where the two halo term will not contribute a lot yet (but actually is that truly the case? 2 arcmin seems like a lot already)

the ratio of 1 halo term sim / 1 halo term analytical is around 12: the 1 halo term from the sims is much higher: and comparing to the size of the data: the sim version is the correct one

Lab 52: literature review

Date: 04/05/2026

Ruppin_2019

abstract:

Cosmological analyses based on surveys of galaxy clusters observed through the Sunyaev Zel'dovich (SZ) effect strongly rely on the mean pressure profile of the cluster population. A tension is currently observed between the cosmological constraints obtained from the analyses of the CMB primary anisotropies and those from cluster abundance in SZ surveys. This discrepancy may be explained by a wrong estimate of the **hydrostatic bias** parameter that links the hydrostatic mass to the true mass of galaxy clusters. However, a variation of both the amplitude and the shape of the mean pressure profile could also explain part of this tension. We analyze the effects of a modification of this profile on the constraints of the 8 and Ωm parameters through the analysis of the SZ power spectrum measured by the Planck collaboration. We choose two mean pressure profiles that are respectively lower and higher than the one obtained from the observation of nearby clusters by Planck. The selection of the parameters of these two profiles is based on the current estimates of the pressure and gas mass fraction profile distributions at low redshift. The cosmological parameters found for these two profiles are significantly different from the ones obtained with the Planck pressure profile. We conclude that a 15% decrease of the amplitude of the mean normalized pressure profile would alleviate the tension observed between the constraints of 8 and Ωm from the CMB and cluster analyses without requiring extreme values of the mass bias parameter.

three causes from the ICM:

1. hydrostatic bias:

x-ray and tsz analyses assume that the IC gas is in hydrostatic equilibrium and use this to infer the cluster mass from the gas pressure profile

however real clusters are not hydrostatic: and have got nonthermal pressure components (these provide extra support against collapse from gravity): if we ignore this pressure we underestimate the true mass

As shown in Salvati et al. (2019), a value of b = 0.38 ± 0.05 is required to cancel the discrepancy between the constraints of 8 and Ωm obtained by the analysis of CMB primary anisotropies and cluster abundance. However, the current estimates of the hydrostatic bias measured with different cluster samples seem to favour a much lower value of the hydrostatic bias parameter. Its average value presented in Salvati et al. (2018) is given by b = 0.2±0.08

2. Y-M relationship:

A second source of systematic effect in the relation between the tSZ observable and the cluster mass may arise from a mass and redshift dependence of the slope of the YM scaling relation.

3. pressure profile

a significant part of the observed discrepancy between the CMB and cluster cosmological parameters may also be due to an evolution of the mean normalized pressure profile of galaxy clusters with mass and redshift. In this case, the tSZ observable measured by assuming that all galaxy clusters share the same normalized pressure profile may also be biased on average.

cosmological tension:

cluster catalogs such as the planck collaboration and the south pole telescope and atacama

cosmology telescope are used to infer the sigma 8 which is the structure amplitude and omega m the matter density: however these disagree with CMB constraints.

main text of the paper:

In this paper, we estimate the implications of a modification of the mean normalized pressure profile of the cluster population on the constraints of the 8 and Ωm parameters by analyzing the angular power spectrum of the tSZ effect measured by the Planck collaboration (Planck Collaboration 2016b). We define two extreme mean normalized pressure profiles that are respectively lower and higher than the one obtained from the observations of 62 nearby clusters by the Planck collaboration (Planck Collaboration et al. 2013). For that purpose, we use the current knowledge on the distributions of normalized pressure profiles and gas mass fraction profiles observed at low redshift (e.g. Eckert et al. 2013, 2019). Three Markov chain Monte Carlo (MCMC) analyses are performed in order to fit the Planck data with a tSZ angular power spectrum model based on these profiles. The cosmological constraints obtained from these are compared

We conclude that a 15% decrease of the amplitude of the mean normalized pressure profile would alleviate the tension observed between the constraints of 8 and Ωm from the CMB and cluster analyses without requiring extreme values of the mass bias parameter.

mccarthy2023flamingoprojectrevisitings8

trying to explain the S8 tension using baryonic feedback with the FLAMINGO simulationsuite

We explore the dependence of the predictions on box size and resolution, cosmological parameters including the neutrino mass, and the efficiency and nature of baryonic 'feedback'. Despite the wide range of astrophysical behaviours simulated, we find that baryonic effects are not sufficiently large to remove the 8 tension. Consistent with recent studies, we find the CMB lensing power spectrum is in excellent agreement with the standard model, whilst the cosmic shear power spectrum, tSZ effect power spectrum, and the cross-spectra between shear, CMB lensing, and the tSZ effect are all in varying degrees of tension with the CMB-specified standard model.

Despite the wide range of astrophysical behaviour simulated, we will show that baryonic effects and their uncertainties encapsulated within the FLAMINGO simulations are not sufficiently large to significantly alter the current S8 discussion.

the dependence of the tsz signal on the simulation box size:

on small l scales, the tsz power spectrum is mostly dependent on the very high mass/ random clusters: (think that Y scales as $M^{5/3} but then the power spectrum is proportional to Y^2$: $so basically M^{10/3}$);

for all simulations we can see less signal then for observations;

also large variance on the smallest l scales;

varying baryonic feedback:

stellar and AGN feedback;

"Our choice of calibration observables is motivated by the fact that the impact of baryon physics on LSS is highly correlated with the baryon fractions of galaxy groups and clusters".

Moser_2021

the missing baryon problem: from cosmological constraints we know how much matter should exist: but observationally at low redshifts did not observe them: most suspected to reside in the WHIM (warm intergalactic medium) or in the filaments.

Schaan_2021

A. Filtering

Compensated aperture photometry (CAP) filters: have varying aperture radius centered around each galaxy; the output of the CAP filter is defined by

$\mathrm{T}(\theta_d) = \int d^2\theta\, \delta T(\theta)\, W_{\theta_d}(\theta)$,

with the filter chosen as:

$$\mathrm{W}_{\theta_d}(\theta) = \begin{cases} 1 & \text{for } \theta < \theta_d, \\ -1 & \text{for } \theta_d \leq \theta \leq \sqrt{2}\,\theta_d, \\ 0 & \text{otherwise.} \end{cases}$$

This corresponds to measuring the integrated temperature fluctuation in a disk with radius theta d and substracting the same signal measured in a concentric ring of the same aread around the disc (the CAP units are microkelvins times arcmin^2)

Because the filter is compensated: i.e the window integrates th area to zero: the filter has the desirable properly that fluctuations with wavelength longer than the filter size will cancel in the substraction: i.e the filter works like : average inside disk - average in the annulus around it: for a contribution which doesnt vary: stay constant across both cap filters and cancels out;

This sig reduces the noise from degree-scale CMB fluctuations and the correlation between the various CAP filter sizes.

B. Stacking:

combining the measured temperatures around each galaxy for a given aperture radius; assume the CAP filter noise is independent from one galaxy to the other

$$\hat{T}_{\mathrm{tSZ}}(\theta_d) = \frac{\sum_i \mathcal{T}_i(\theta_d)/\sigma_i^2}{\sum_i 1/\sigma_i^2}$$

1. stacking is a two point correlation function in real space: This approach is formally equivalent to measuring the cross-power spectrum of the temperature map with a template map:

2. advantages of cross-power spectra over the configuration space / real space stacking:

-> have more independent l bins: covariance matrix is simpler and easier to analyze; -> for CAP aperture: the profiles at adjacent radii are not independent: off diagonal terms in the correlation matrix

the two approaches have the same signal to noise ratio:

from parsevals theorem: no information is lost or gained when doing a fourier transform;

3. advantages of the configuration space over cross power spectra:

this is much more intuivite to talk about the shape of the profile: as the whole point is inferring P(r)

C. covariance matrices:

for an ideal map, i.e uniform noise, gaussian statistics could just infer the covariance from the power spectrum of the map; however the ACT maps are messy.

instead do bootstrap: change the galaxy sample: recompute the stacked profile and then measure the scatter

D. Dust contamination to tSZ and kSZ:

Comes from the spatial correlation of the thermal emission from dust in the galaxy sample or galaxies;

positive signal to both f150 and f90 maps: which partially cancel the tSZ signal and thus bias the inference about the circumgalactic gas

the tsz signal is negative under 217 GHz; wher3eas dust goes as a blackbody distribution

f150, f90: different frequency maps from ACT; each frequency has different contributions (i.e from CMB, tsz and dust etc)

the fiducial tSZ results in the paper are obtained from the CIB-deprojected ILC y map:

ILC - internal linear combination: combine multiple frequency maps and choose coefficients such to preserve the tSZ signal and

Sehgal_2010

abstract:

create realistic mock microwave sky maps for ACT-like experiments N-body simulations -> paint baryonic gas -> mock y maps

Battaglia_2012

Discussion about choosing the cut-off radius for the tSZ signal

We also find that there are contributions to the average pressure profile at larger radii from substructure and nearby clusters, which cause relative deviations from the mean profile $> 5\%$. In a companion paper, we also show that substructure affects the kinetic support in cluster outskirts and the shape of the ICM shape at similar radii (Battaglia et al. 2011). In these regions (redshift dependent, but typically 2R200) Pfit often deviates from $P^{-}th$ by more than 5%. We chose not to model this behavior because of two reasons. First, the problem of double-counting SZ flux: the large volume contained within the radius that contains 95% of the total SZ flux, $r < 4R200$, necessarily leads to overlapping volumes of neighboring clusters, especially at high-redshift.

Second, the total SZ flux of an increasing pressure profile, scaled by x3, does not converge and an arbitrarily chosen radial cutoff would substantially contribute to the resulting power of the tSZ power spectrum. Because we weight by the variance within radial bins, these contaminated regions are naturally down-weighted in the profile fits.

Battaglia_2012:

Cut-off radius discussion:

even if the tsz ignal drops off quite quickly at large radii: the volume scales up quickly: which can still diverge in total;

at large radii: the pressure belongs to neighbouring halos/ filaments/ infalling structure/ messy

the 4 $R_200 or equivalently 4 times r98 is a scale relevant for baryonic feedback$;

Liu_2025

If the true tSZ profile was known a priori and we were only interested in measuring the amplitude of the known profile, a matched filter would be the minimum variance unbiased linear estimator. However, the tSZ profile is not known and the goal of this study is to measure this profile. Therefore we use a set of filters and we vary the size of d between 1 and 6 arcmin, which

corresponds to approximately 0.5-4 virial radii, the physical scales relevant for feedback and baryonic effects.

Hill2014

the integral is divergent for tsz with volume:

The primary obstacle is the logarithmic divergence of the enclosed mass in the NFW profile as r $\rightarrow$ [77]. By testing various values of the outer cut-off in Eq. (3.10), we find that rout = 1.5rvir leads to better than 5% agreement between our halo model calculation of the CMB lensing potential power spectrum and the linear theory calculation

PhysRevD.97.083501

Colin Hill two halo term paper:

questions to boryana:

1. why didn't they compare to the flamingo sims in her evidence paper 2. what specific cosmology should we use in case we rerun the code

Date: 06/05/2026